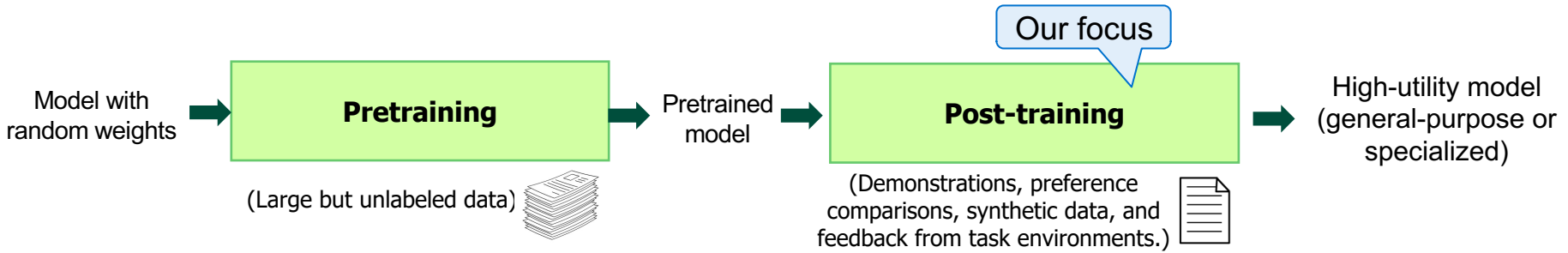
Post-training

CSCI 601-771 (NLP: Self-Supervised Models)

https://self-supervised.cs.jhu.edu/fa2026/

Post-training Language Models

- There is a mismatch between what **pre-trained** models can do and what we want.
- Addressing this gap is the focus of “alignment” research.
- Let’s take a deeper look into what “alignment” is about.



What do people actually do?

Methods SFT RL RLHF PPO GRPO

Feedback Human preferences Verifiable rewards Model-based (reward models, LLM graders, teachers)

Model	Year	Pipeline	Feedback source	Data
Llama 2-Chat	2023	SFT → RLHF (rejection sampling + PPO)	Human preferences → reward models	27.5K SFT
Llama 3 / 3.1	2024	Rejection sampling → SFT → DPO	Human preferences; code execution checks	Heavy synthetic SFT
DeepSeek-R1	2025	SFT → RL (GRPO) → SFT → RL	Verifiable rewards + preference model	800K SFT
Nemotron 3 Ultra	2026	SFT → RL (GRPO) → multi-teacher on-policy distillation + RLHF	Verifiable rewards; preference model; teacher models	10M SFT; 1M RL tasks
Kimi K3	2026	SFT → 9 RL experts → multi-teacher on-policy distillation	Verifiable rewards; LLM graders; teacher models	Synthetic agent tasks
GLM-5.3	2026	Scaled RL only (SAO)	Verifiers on executable environments	Long-horizon environments
MiMo-V2.6-Pro	2026	SFT → mixed RL (GRPO) → on-policy distillation	Test pass × LLM quality checklists	7K+ RL environments

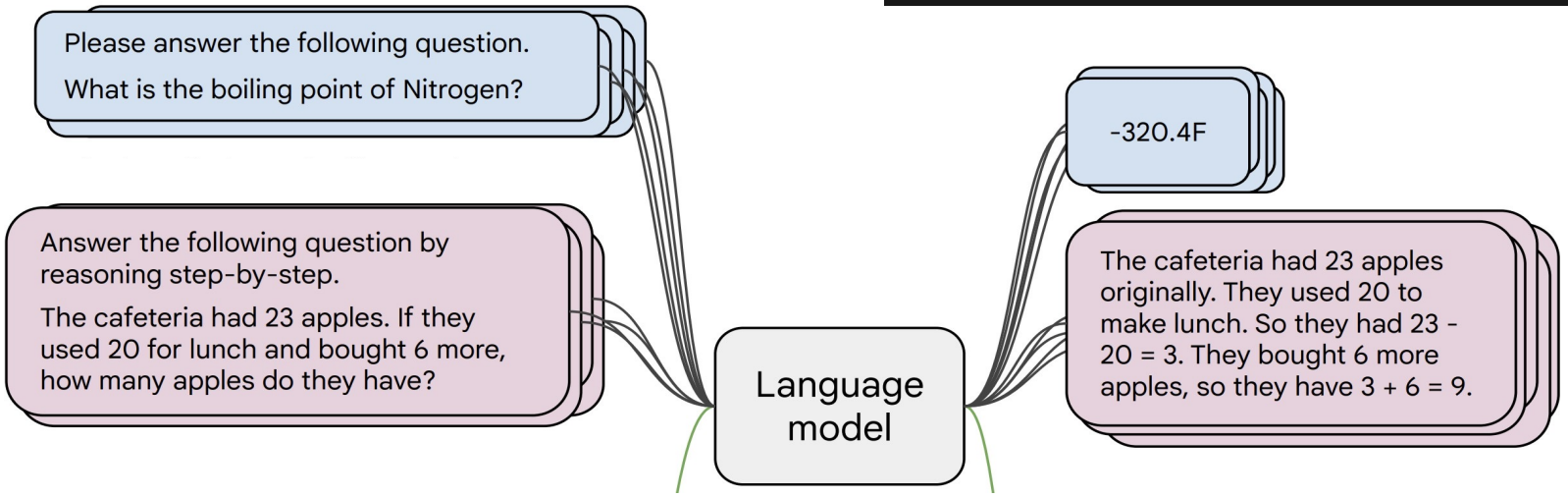
We will see some of these!

Instruction-tuning

Instruction-tuning

- Instruction tuning trains a pretrained LM to generate a demonstrated response conditioned on an instruction and any accompanying context.
- We typically compute the loss on response tokens; instruction and context tokens are provided as conditioning information.

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim D}\left[\sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t})\right].$$

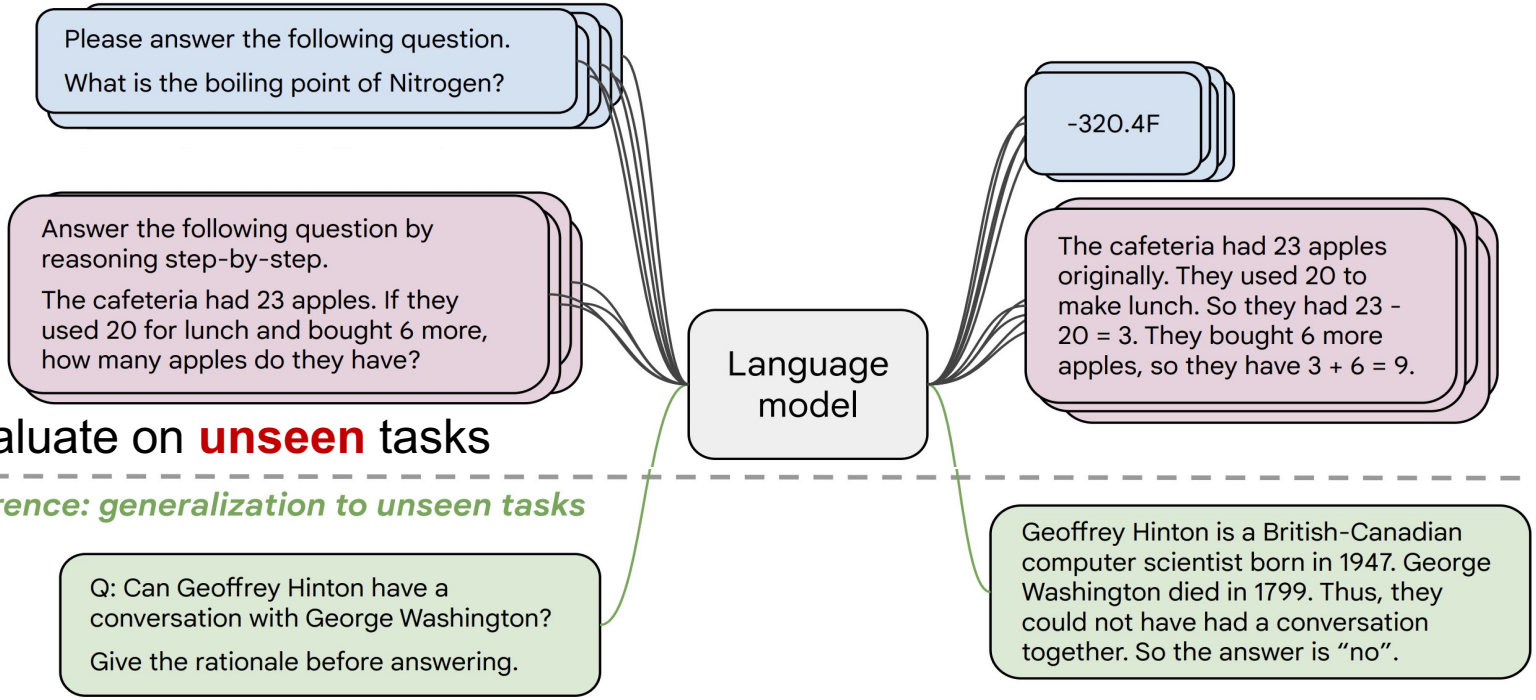


Instruction-tuning

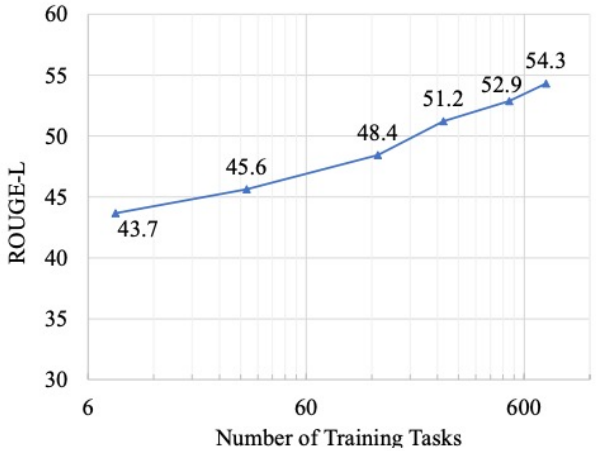
[Weller et al. 2020. Mishra et al. 2021;
Wang et al. 2022; Sanh et al. 2022;
Wei et al., 2022, Chung et al. 2022, many others]

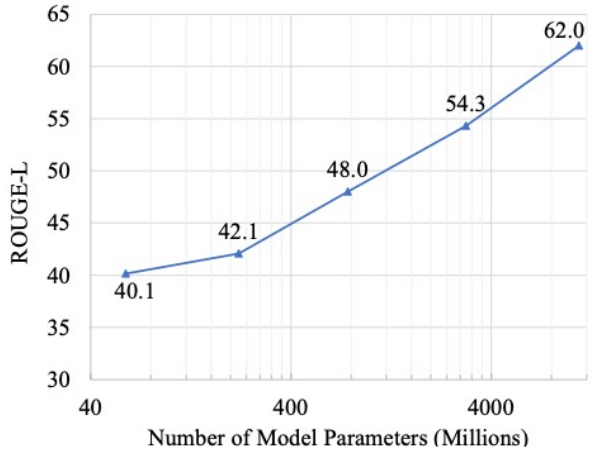
- Task generalization means evaluating on task types excluded from training.

1. Collect examples of (instruction, output) pairs across many tasks and finetune LM.



Scaling Instruction-Tuning





Linear growth of model performance
with exponential increase in observed tasks and model size.

Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks, Wang et al. 2022

Cross-Task Generalization via Natural Language Crowdsourcing Instructions (Mishra et al., 2022)

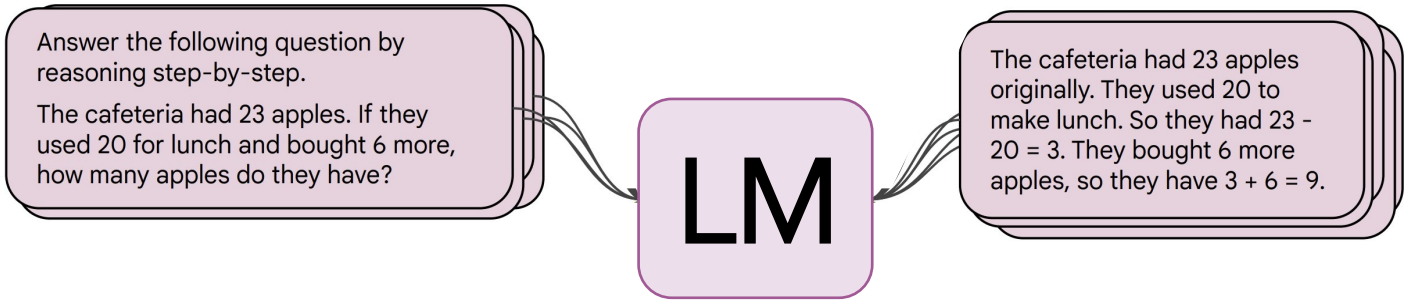
Pulse check

- **Q1:** Does instruction tuning require a different objective from pretraining?
- **A1:** Mostly no—both use next-token prediction. SFT typically computes loss on response tokens, conditioned on the instruction and context.

- **Q2:** If a model succeeds on new examples of familiar tasks, has it demonstrated unseen-task generalization?
- **A2:** No. Unseen-task evaluation holds out entire task types, not just individual examples.

Limits of Instruction-Tuning

1. Difficult to collect diverse data.
2. Resulting models may not be good at open-ended generation tasks.
   - SFT increases the likelihood of demonstrated responses. => Its effectiveness therefore depends strongly on the quality and coverage of the demonstrations.



Limits of Instruction-Tuning

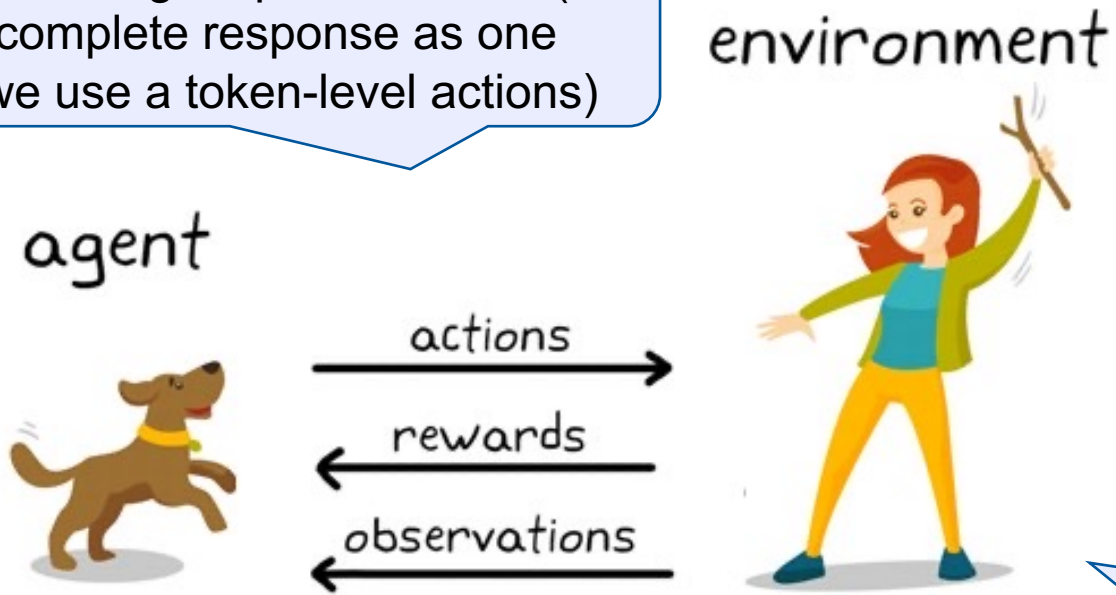
1. Difficult to collect diverse data.
2. Resulting models may not be good at open-ended generation tasks.
   - SFT increases the likelihood of demonstrated responses. => Its effectiveness therefore depends strongly on the quality and coverage of the demonstrations.
3. SFT can encourage unsupported factual answers when demonstrations require knowledge the model does not reliably possess.
   - Labeled data is collected agnostic to the LM's knowledge (off-policy) => there might be a mismatch between labeled data and LM knowledge.

Does Fine-Tuning LLMs on New Knowledge Encourage Hallucinations?

Reinforcement Learning

Reinforcement Learning: Intuition

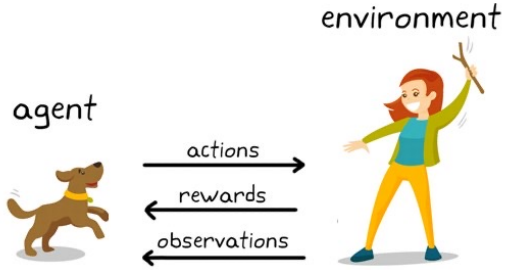
Action here: generating responses/token (for now, treat a complete response as one action. Later, we use a token-level actions)



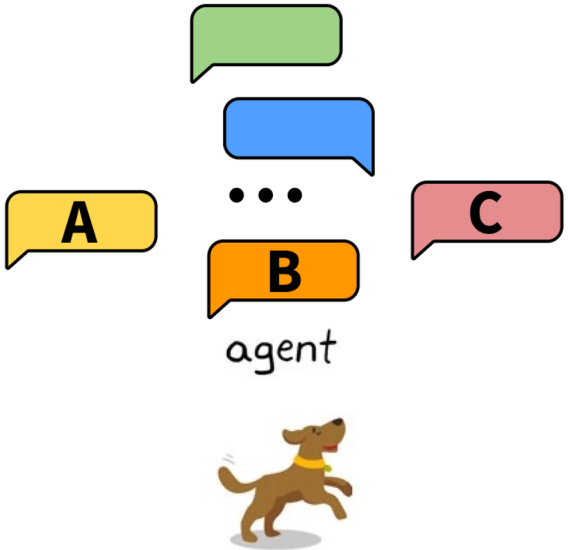
[figure credit]

Reward here: whether humans liked the generation (sequence of actions=tokens)

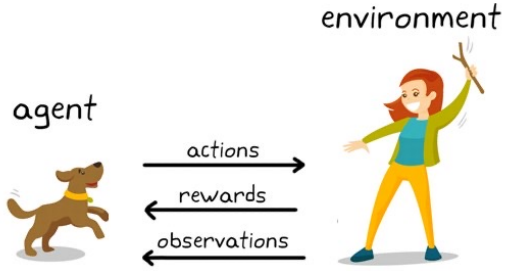
Intuition



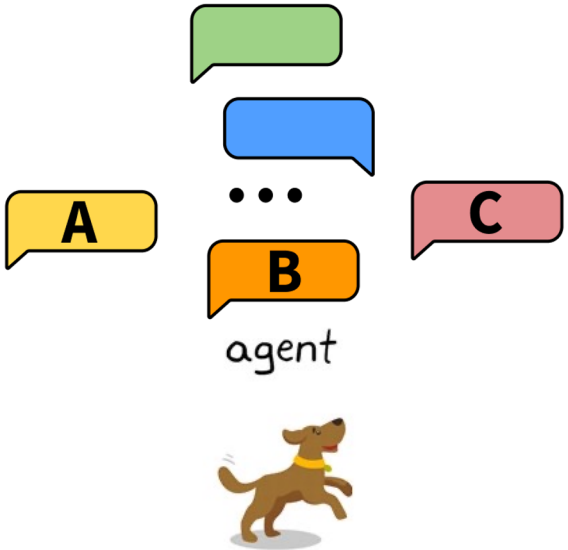
- The LM generates candidate responses to a conversation.
- Human compare those candidates

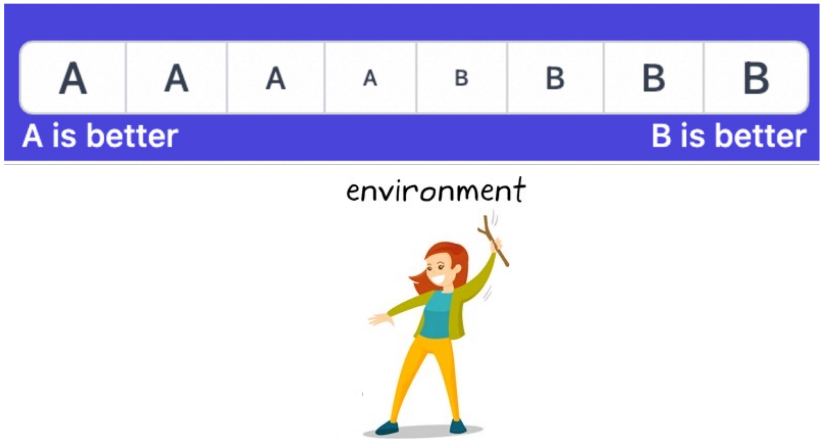


Intuition



- Scoring interface: Likert scale or rankings
  - E.g., each annotation identifies a preferred vs. a rejected response to the same prompt.
  - Preferences depend on the annotation rubric and may differ across annotators.





Reinforcement Learning: Formalism

- **State s_t:** the prompt and generated prefix.
- **Action a_t**: the next token.
- **Episode:** the agent (policy) generates actions (tokens) until a stopping condition.
- **Transition:** append the generations to the prefix to obtain a new state of the world.
- **Reward:** The environment rewards the action, only after the response is complete.
- We need to figure out: (1) reward function and (2) the policy function.



s_t: state (context)
r_t: reward
a_t: actions (new sentences)
p_θ: policy (decision-maker)

Reinforcement Learning from Human Feedback



- Imagine a reward function: $R(s; \text{prompt}) \in \mathbb{R}$ for any output s to a prompt.
- The reward is higher when humans prefer the output.
- Good generation is equivalent to finding reward-maximizing outputs:

$$\mathbb{E}_{\text{prompt}\sim D}\,\mathbb{E}_{\hat{s}\sim\pi_\theta}\left[R(\hat{s}; \text{prompt})\right]$$

Expected reward over the course of sampling from our policy (generative model)

$\pi_\theta(s)$ is a pre-trained model with params θ we would like to optimize (policy function)

- On the notation:
  - "$\mathbb{E}$" here is an empirical expectation (i.e., average).
  - "$\sim$" indicates sampling from a given distribution.

Reinforcement Learning from Human Feedback



- Imagine a reward function: $R(s; \text{prompt}) \in \mathbb{R}$ for any output s to a prompt.
- The reward is higher when humans prefer the output.
- Good generation is equivalent to finding reward-maximizing outputs:

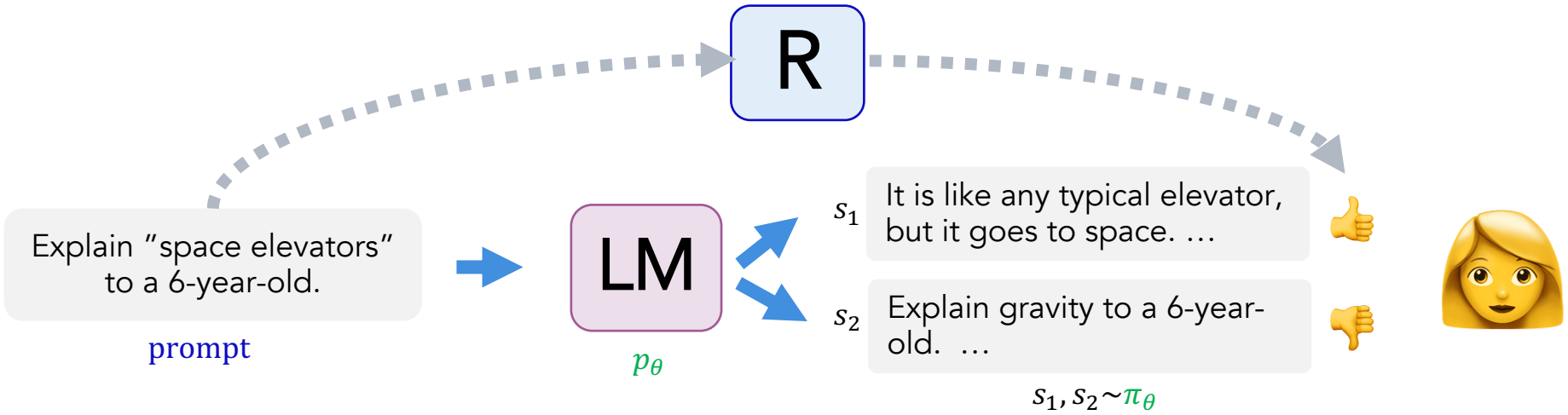
$$\mathbb{E}_{\text{prompt}\sim D}\,\mathbb{E}_{\hat{s}\sim\pi_\theta}\left[R(\hat{s}; \text{prompt})\right]$$

- We optimize **average** reward over a distribution of prompts and responses *sampled from the current policy*.

- What we need to do:
  - (1) Estimate the reward function $R(s; \text{prompt})$.
  - (2) Find the best generative model π_θ that maximizes the expected reward:

Step 1: Estimating the Reward R

- Humans look at alternative generations and indicate their preference.



Which of these two responses is more helpful?

What are the steps for making a simple cake?

1. *Preheat oven to 350°F (175°C).*
2. *Grease and flour a cake pan.*
3. *In a bowl, combine 2 cups flour, 1.5 cups sugar, 3.5 tsp baking powder, and a pinch of salt.*
4. *Add 1/2 cup butter, 1 cup milk, and 2 tsp vanilla; mix well.*
5. *Beat in 3 eggs, one at a time.*
6. *Pour batter into the pan.*
7. *Bake for 30-35 minutes or until a toothpick comes out clean.*
8. *Let cool, then frost or serve as desired.*

What are the steps for making a simple cake?

1. *Warm up the oven.*
2. *Grease a cake pan.*
3. *Blend dry ingredients in a bowl.*
4. *Incorporate butter, milk, and vanilla.*
5. *Mix in the eggs.*
6. *Pour into the prepared pan.*
7. *Bake until golden brown.*
8. *Add frosting if desired.*

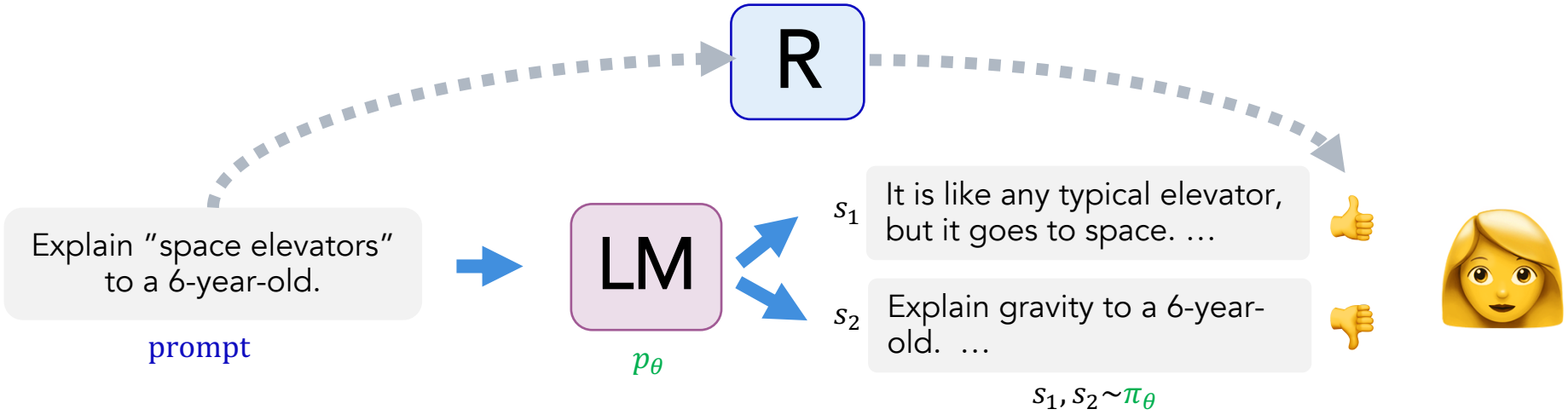
[Example from Eric Mitchell]

Step 1: Estimating the Reward R

$$P(s^+ > s^- \mid \text{prompt}) := \sigma\big(R(s^+; \text{prompt}) - R(s^-; \text{prompt})\big)$$

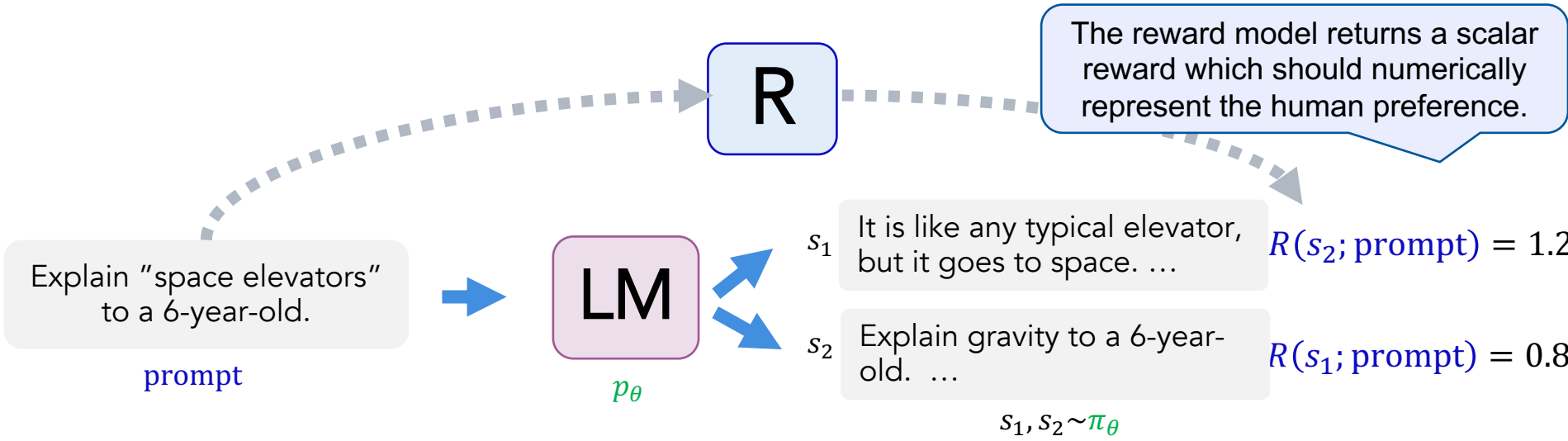
"winning" sample

"losing" sample



Step 1: Estimating the Reward R

$$J_{RM}(\phi) = \mathbb{E}_{\text{prompt}\sim D}\mathbb{E}_{(s^+, s^-)\sim\pi_\theta}\left[\log\sigma\left(R(s^+;\text{prompt}) - R(s^-;\text{prompt})\right)\right]$$



Step 1: Estimating the Reward R

$$J_{RM}(\phi) = \mathbb{E}_{\text{prompt}\sim D}\,\mathbb{E}_{(s^+, s^-)\sim\pi_\theta}\left[\log\sigma\left(R(s^+;\text{prompt}) - R(s^-;\text{prompt})\right)\right]$$

```
loss_fn = nn.BCEWithLogitsLoss()  # For preference classification

for epoch in range(epochs):
    model.train()
    total_loss = 0
    for a, b, label in dataloader:
        a, b, label = a.to(device), b.to(device), label.to(device).float()

        reward_a = model(a)
        reward_b = model(b)

        logits = reward_a - reward_b
        loss = loss_fn(logits, label)
```

Where did the “sigmoid” go??

https://docs.pytorch.org/docs/2.15/generated/torch.nn.BCEWithLogitsLoss.html

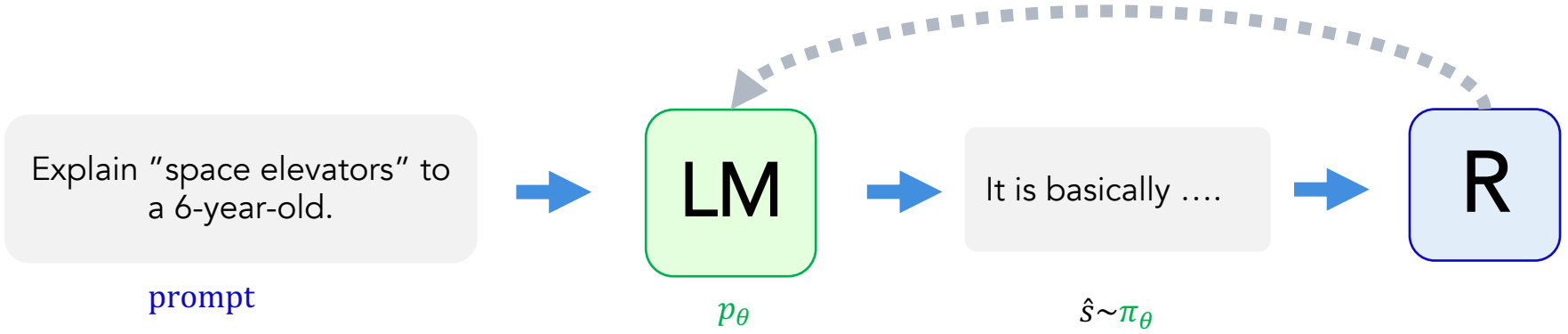
Pulse check

- **Q:** Do we need humans to assign numerical rewards to train a reward model?
- **A:** No. Pairwise preferences are enough: train the model to score the preferredresponse higher than the rejected response.

Step 2: Optimizing the Policy Function

- **Policy function** := The model that makes decisions (here, generates responses)
- How do we change our LM parameters θ to maximize this?

$$J(\theta) = \mathbb{E}_{\text{prompt}\sim D}\mathbb{E}_{\hat{s}\sim\pi_\theta}[R(\hat{s};\text{prompt})]$$



Step 2: Optimizing the Policy Function

- **Policy function** := The model that makes decisions (here, generates responses)
- How do we change our LM parameters θ to maximize this?

$$\mathrm{J}(\theta) = \mathbb{E}_{\text{prompt}\sim D}\,\mathbb{E}_{\hat{s}\sim\pi_\theta}[R(\hat{s};\text{prompt})]$$

- Let's try doing gradient ascent!

$$\theta_{t+1} \leftarrow \theta_t + \alpha\,\nabla_{\theta_t}\mathrm{J}$$

How do we estimate this gradient?

Notice that R is not directly dependent on θ. (You can't compute its grad with respect to θ)

- The response is sampled through discrete token choices, so ordinary backpropcannot differentiate through the sampled text.
- So how do we estimate these gradients??

Policy Gradient [Williams, 1992]

- How do we change our LM parameters θ to maximize this?

$$\mathrm{J}(\theta) = \mathbb{E}_{\text{prompt}\sim D}\,\mathbb{E}_{\hat{s}\sim\pi_\theta}[R(\hat{s};\text{prompt})]$$

- Let's try doing gradient ascent!

$$\theta_{t+1} \leftarrow \theta_t + \alpha\,\nabla_{\theta_t}\mathrm{J}$$

- With a bit of math, this can be approximated as Monte Carlo samples from π_θ:

$$\nabla_\theta \mathrm{J} \approx \widehat{\nabla_\theta \mathrm{J}} \coloneqq \frac{1}{|\text{samples}|} \sum_{\text{prompt}\sim D} \sum_{s_i\sim\pi_\theta(\text{prompt})} R(s_i;\text{prompt})\,\nabla_\theta \log \pi_\theta(s_i;\text{prompt})$$

- This is "policy gradient", an approach for estimating and optimizing this objective.

Derivations (check it later in your own time!)

- Let's compute the gradient (ignore expectation over prompts, without loss of generality):

Def. of "expectation"

Gradient distributes over sum

$$\nabla_\theta \mathrm{J} = \mathbb{E}_{\hat{s}\sim\pi_\theta}[R(\hat{s};\text{prompt})] \approx \nabla_\theta \sum_s \pi_\theta(s) R(s;p) = \sum_s R(s;p)\,.\,\nabla_\theta \pi_\theta(s)$$

- Log-derivative trick $\nabla_\theta \pi_\theta(s) = \pi_\theta(s)\,.\,\nabla_\theta \log \pi_\theta(s)$ to turn sum back to expectation:

$$\nabla_\theta \mathrm{J} = \sum_s R(s;p)\pi_\theta(s)\,\nabla_\theta \log \pi_\theta(s)$$

- Approximate this expectation with Monte Carlo samples from $p_\theta(s)$:

$$\nabla_\theta \mathrm{J} \approx \frac{1}{n}\sum_{i=1}^{n} R(s;p)\,\nabla_\theta \log \pi_\theta(s)\,. \qquad s \sim \pi_\theta(s)$$

Bonus

Policy Gradient [Williams, 1992]

- This gives us the following update rule:

Note, $R(s; \text{prompt})$ could be any arbitrary, non-differentiable reward function that we design.

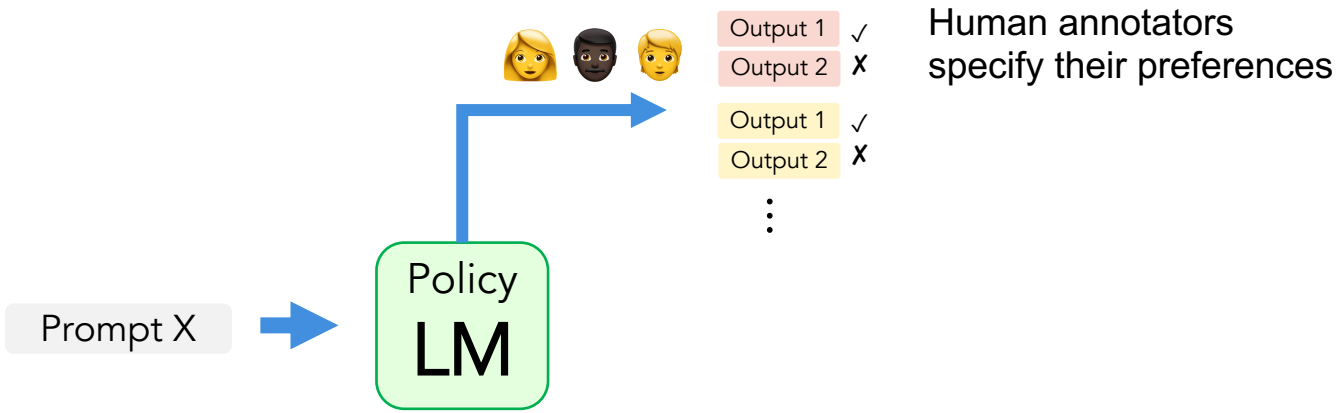
$$\theta_{t+1} \leftarrow \theta_t + \alpha \frac{1}{|\text{samples}|} \sum_{\text{p} \in \text{prompts}} \sum_{s_i \sim \pi_\theta(\text{p})} R(s_i; \text{p}) \nabla_\theta \log \pi_\theta(s_i; \text{p})$$

- The reward supplies the coefficient on the sampled response's log-probability gradient.
  - If $R(s; p)$ is large, we take proportionately large steps to maximize $\pi_\theta(s)$
  - If $R(s; p)$ is small, we take proportionately small steps to maximize $\pi_\theta(s)$

This is why it's called "reinforcement learning":
we reinforce good actions, increasing the chance they happen again.

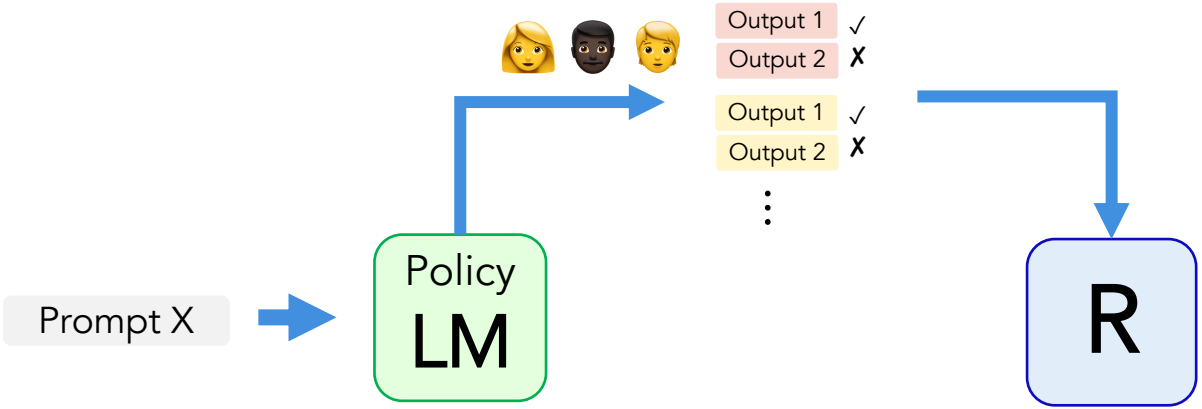
Putting it Together

- Select prompts that elicit responses that cover the desired behaviors + failure modes.
- Present multiple outputs to human annotators and ask them to rank them.



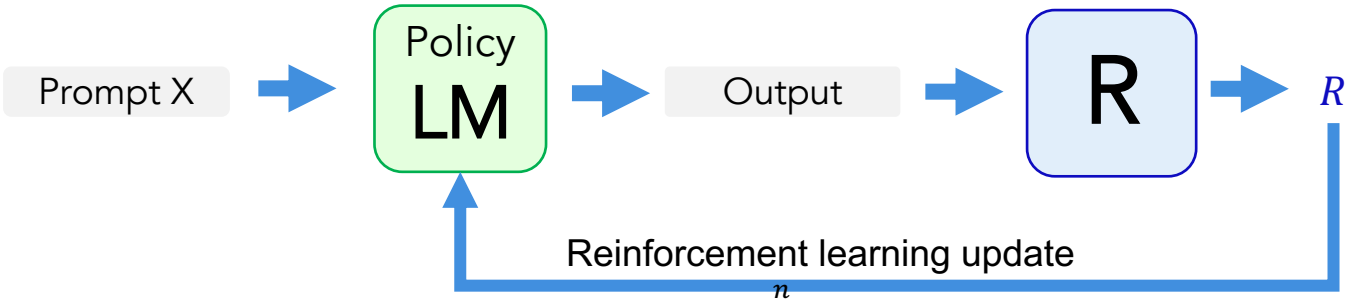
Putting it Together (2)

- Using this data, we can train a reward model
- The reward model returns scalars which should numerically represent human preferences.
- Hold out prompts and preference comparisons to test whether the reward model generalizes beyond its training data.



Putting it Together (3)

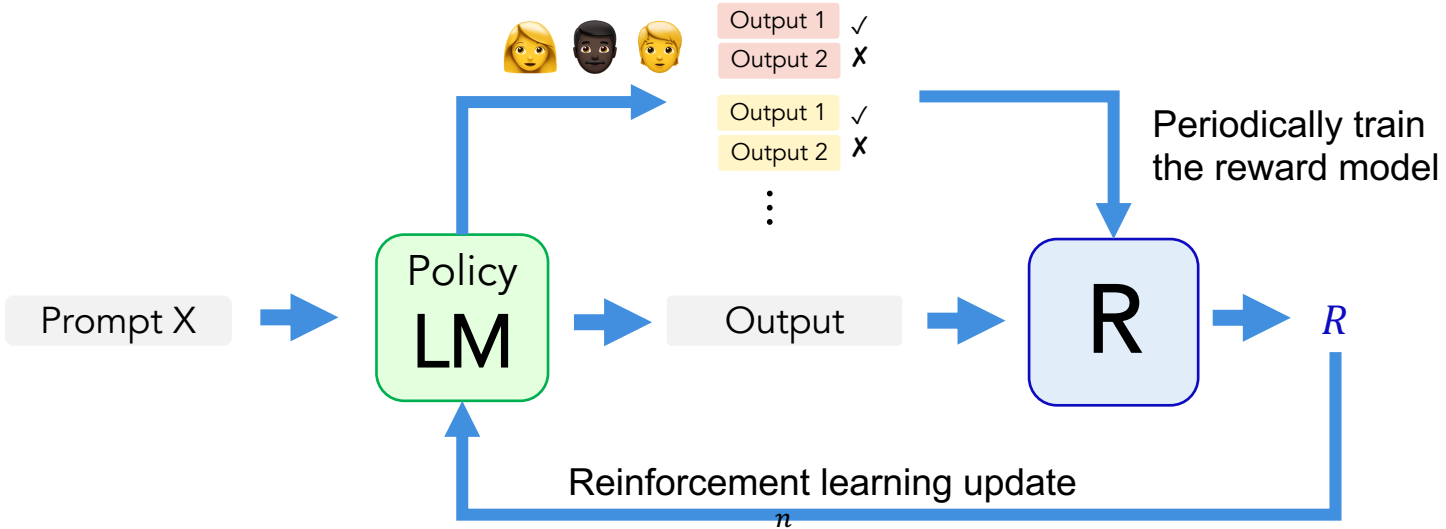
- We want to learn a policy (a Language Model) that optimizes against the reward model



$$\theta_{t+1} \leftarrow \theta_t + \alpha \sum_{i=1}^{n} R(s_i; \mathrm{p})\, \nabla_\theta \log \pi_\theta(s_i; \mathrm{p})$$

Putting it Together (4)

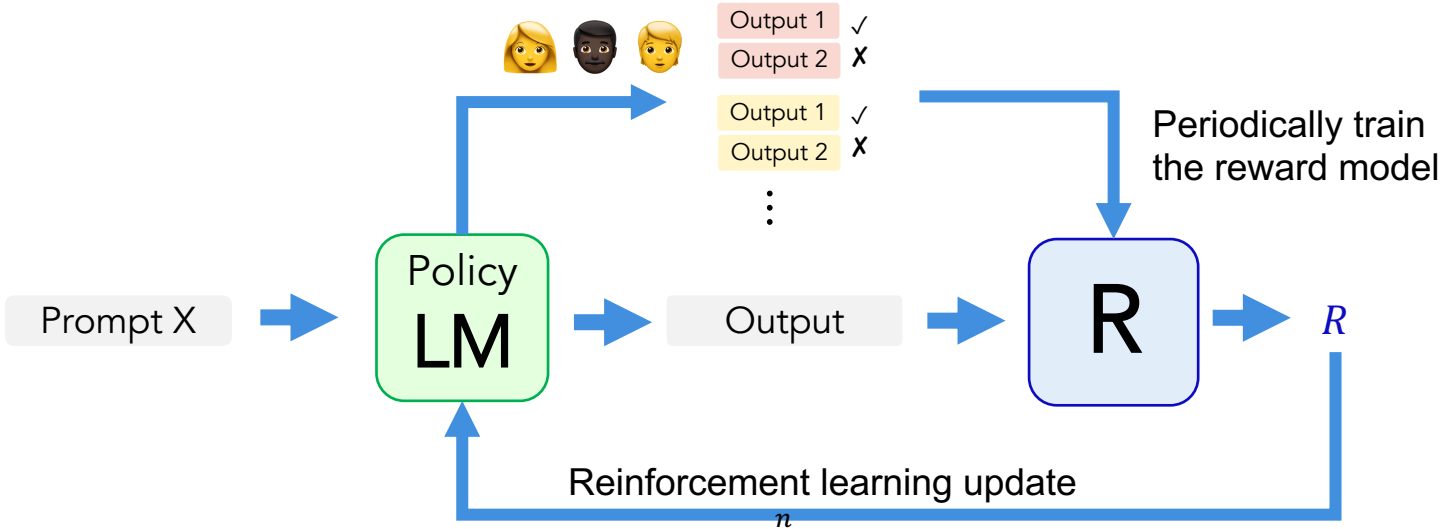
- Optional iterative extension: collect preferences on responses from the updated policy and retrain the reward model with more samples and human feedback.
  - As the policy changes, its responses may differ from those used to train the original reward model. Fresh comparisons can help address this distribution shift.



$$\theta_{t+1} \leftarrow \theta_t + \alpha \sum_{i=1}^{n} R(s_i; \mathrm{p})\, \nabla_\theta \log \pi_\theta(s_i; \mathrm{p})$$

One missing ingredient

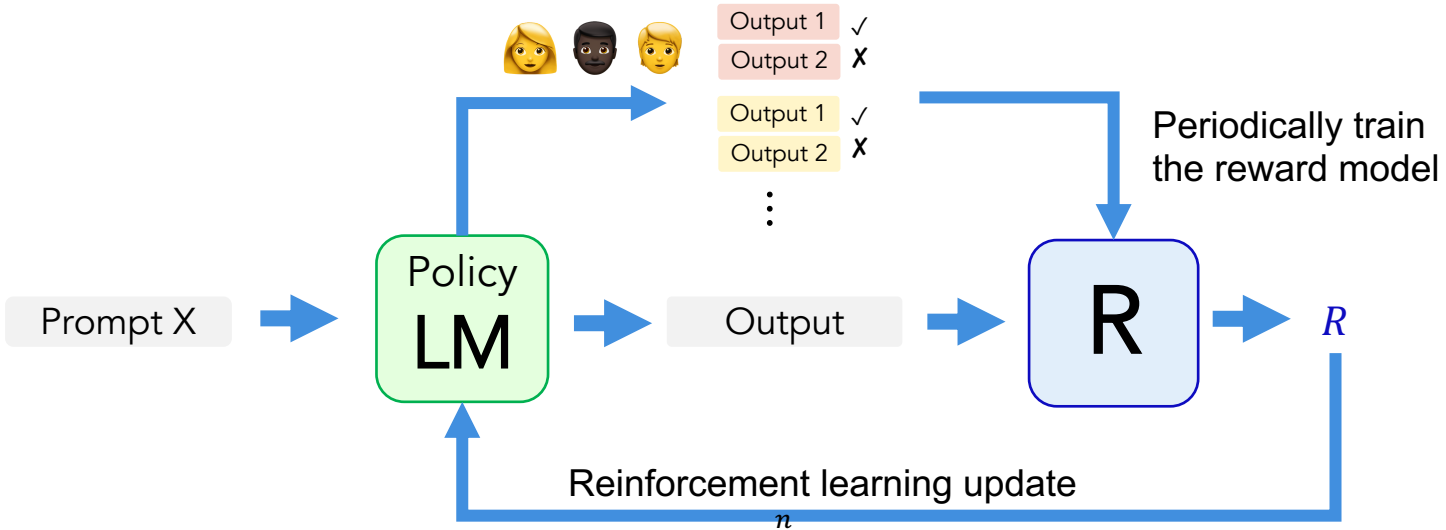
- It turns out that this approach doesn't quite work. (Any guesses why?)
- An imperfect reward model creates an opportunity for "reward hacking": policy optimization can increase predicted reward without improving actual response quality.



$$\theta_{t+1} \leftarrow \theta_t + \alpha \sum_{i=1}^{n} R(s_i; \mathrm{p})\, \nabla_\theta \log \pi_\theta(s_i; \mathrm{p})$$

One missing ingredient

- "Reward hacking" can produce fluent, plausible responses.
  - For example, excessive verbosity, flattery, or confident but unsupported answers that receive high predicted rewards.



$$\theta_{t+1} \leftarrow \theta_t + \alpha \sum_{i=1}^{n} R(s_i; \mathrm{p})\, \nabla_\theta \log \pi_\theta(s_i; \mathrm{p})$$

Regularizing with Pre-trained Model

- **Solution:** add a penalty term that penalizes too much deviations from the distribution of the pre-trained LM.

$$\hat{R}(s;p) \coloneqq R(s;p) - \beta \log \left(\frac{\pi_\theta(s)}{\pi^{\text{ref}}(s)} \right)$$

- This prevents the policy model from diverging too far from the pretrained model.
  - $\pi_\theta(s) \gg \pi^{\text{ref}}(s)$: Pay an *explicit* price.
  - $\pi_\theta(s) \ll \pi^{\text{ref}}(s)$: Sampling s becomes unlikely.
- For responses sampled from the current policy, the expected log-ratio (above) equals the KL divergence from the reference policy.
  - KL regularization penalizes distribution shift. It does not guarantee that reward hacking or undesirable behavior will disappear.

Tangent: KL Divergence

- KL divergence := measures how different one probability distribution is from another.

$$D_{\mathrm{KL}}(P, Q) = \mathbb{E}_{x \sim P}\left[\log \frac{P(x)}{Q(x)}\right] = \sum_{x} P(x) \log \frac{P(x)}{Q(x)}$$

- Properties:
  - $D_{KL} \geq 0$
  - $D_{KL}(P, Q) = 0$ if and only if P = Q
  - Asymmetric: $D_{KL}(P, Q) \neq D_{KL}(Q\ P)$
  - Larger KL generally means greater disagreement between the distributions.

Tangent: KL Divergence

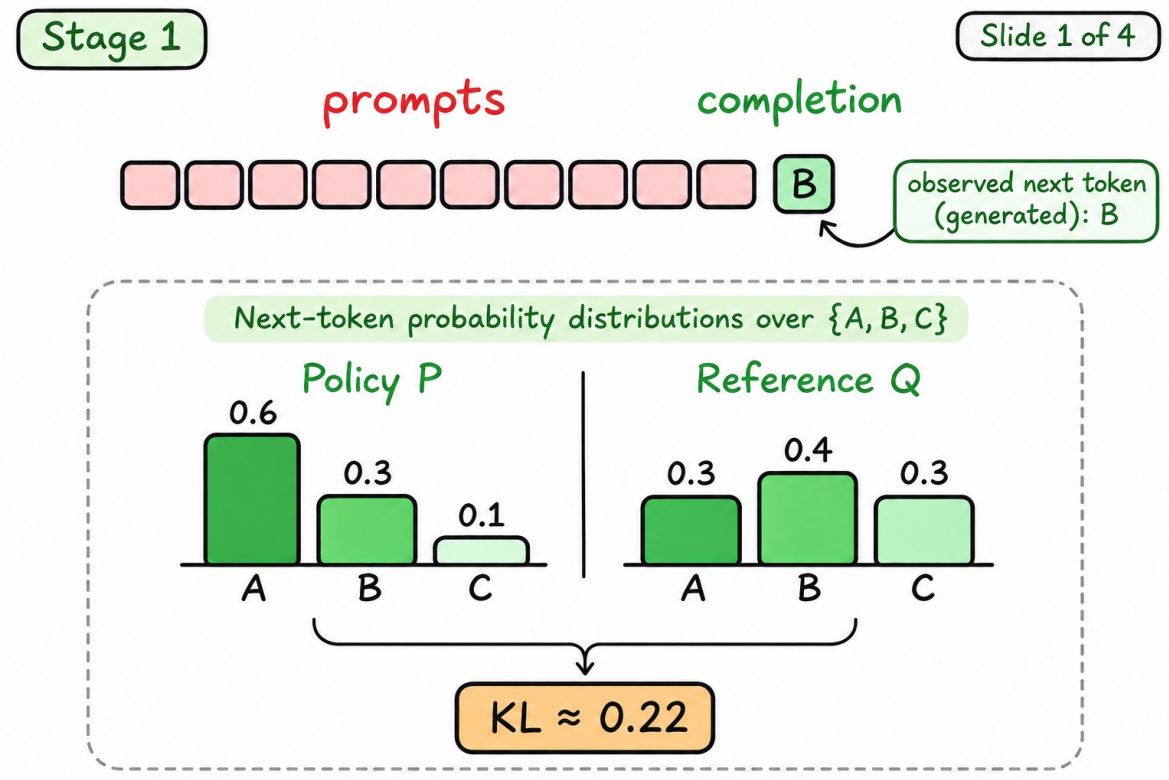
- KL divergence := measures how different one probability distribution is from another.

$$D_{\mathrm{KL}}(P, Q) = \mathbb{E}_{x \sim P}\left[\log \frac{P(x)}{Q(x)}\right] = \sum_{x} P(x) \log \frac{P(x)}{Q(x)}$$

- Demo: https://claude.ai/artifact/NyhNkvmeYKg6tKhFiMkh2E

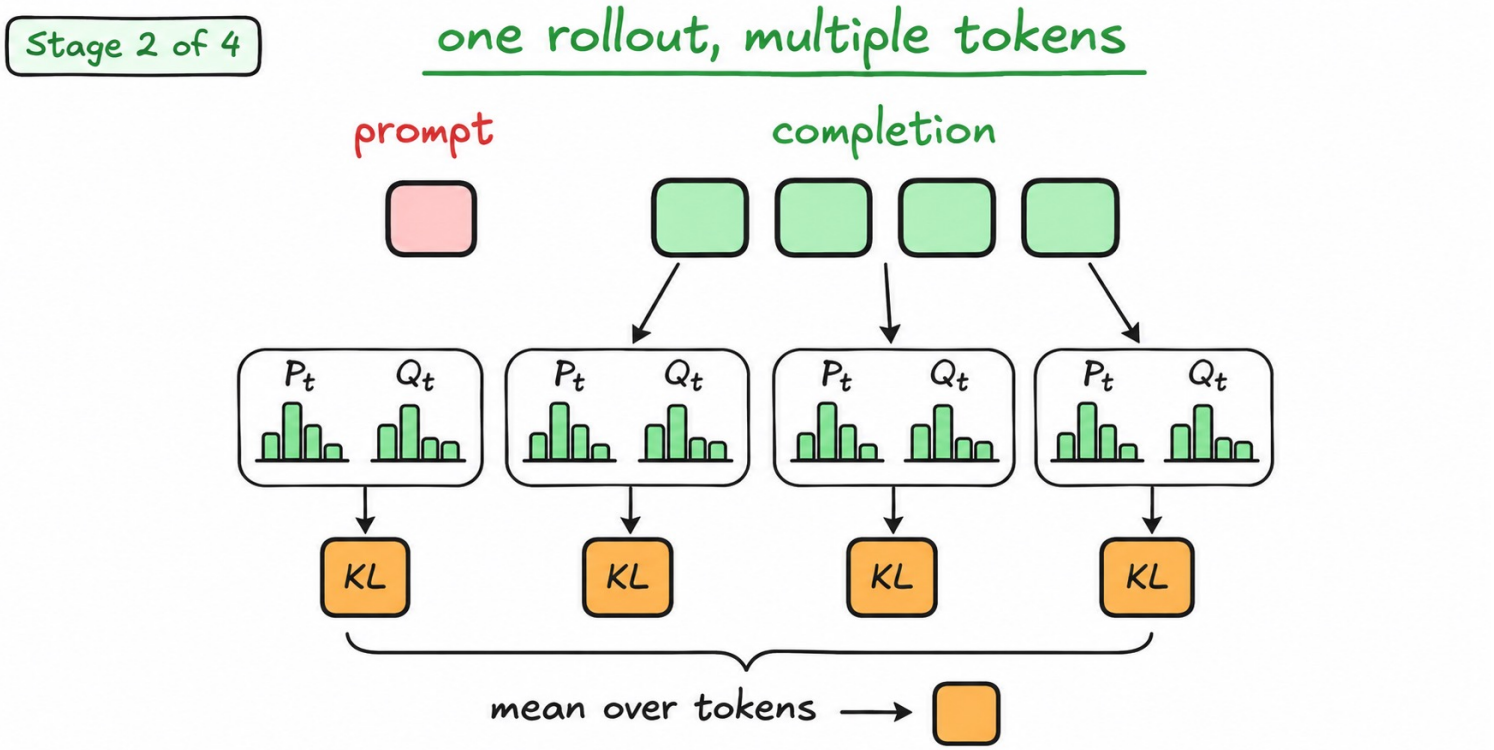
KL over language responses

- 1. Compare two next-token distributions and compute one KL value.



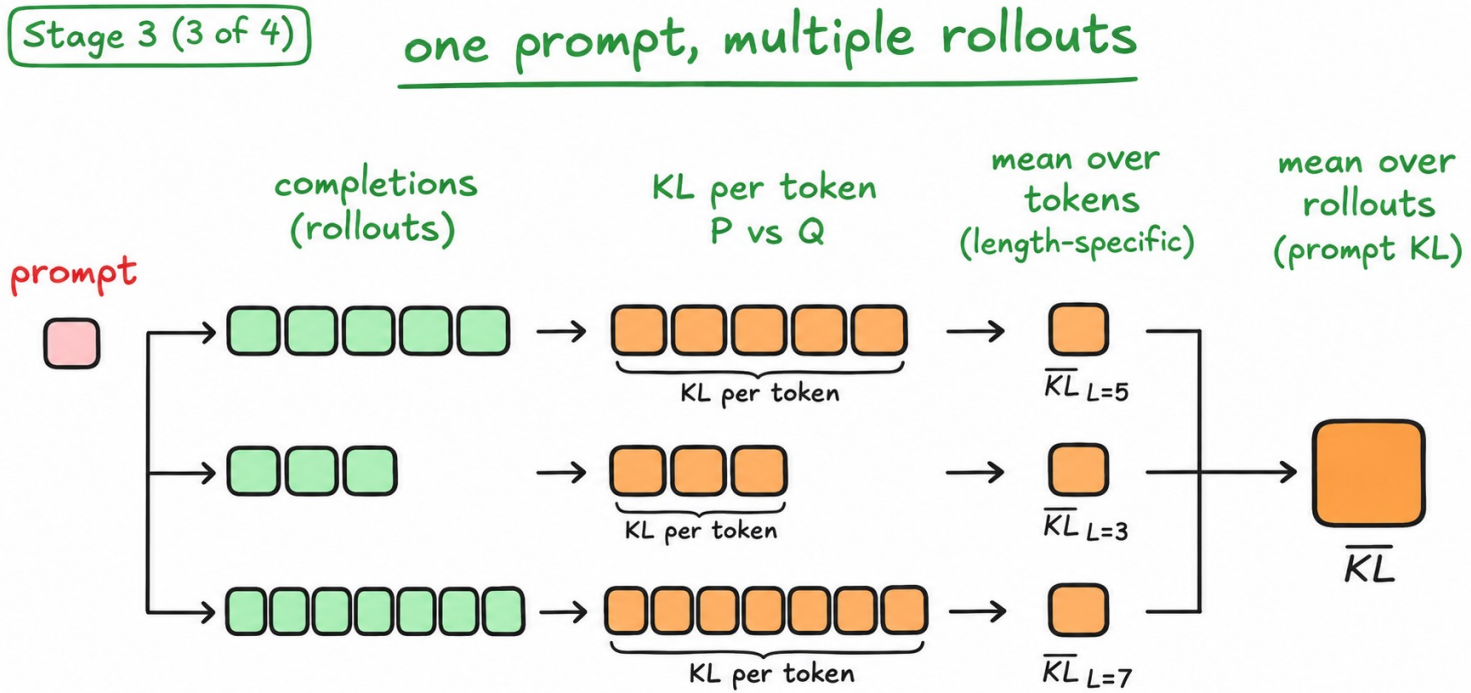
KL over language responses

- 2. Compute KL at each token position, then average across tokens.



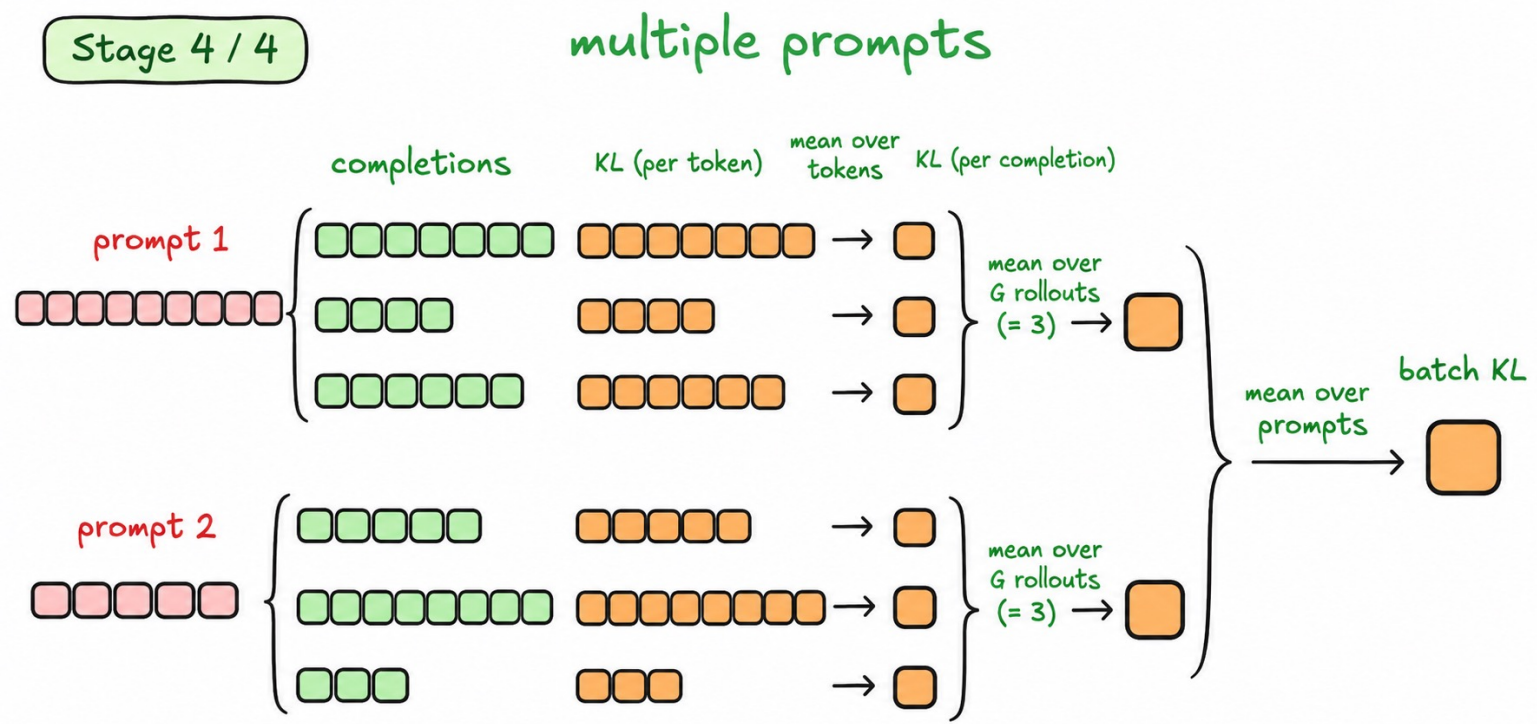
KL over language responses

- 3. Repeat for several completions of the same prompt, then average their KL values.



KL over language responses

- 4. Avg the completion-level KL values within each prompt, then average across prompts.



Putting it All Together:

RLHF as a Basic Policy Gradient

1. Initialize the policy π_θ from an SFT model and choose a reference policy π^{ref}.
2. Train a reward model $R(s;p)$ from preference comparisons.
3. Iterate:
    1. Repeatedly sample responses and update the policy π_θ to increase reward while penalizing departure from the reference:

$$\hat{R}(s;p) \coloneqq R(s;p) - \beta \log\left(\frac{\pi_\theta(s)}{\pi^{\text{ref}}(s)}\right)$$

$$\theta_{t+1} \leftarrow \theta_t + \alpha \frac{1}{|\text{samples}|} \sum_{\text{p} \in \text{prompts}} \sum_{s_i \sim \pi_\theta(\text{p})} R(s_i;\text{p})\, \nabla_\theta \log \pi_\theta(s_i;\text{p})$$

    2. Occasionally repeat repeat 2-3 to update the reward model.

Stiennon et al. Learning to summarize from human feedback, 2020

The overall recipe 👨‍🍳

- SFT learns from demonstrated responses.
- Preference/outcome optimization (e.g. RLHF) learn from evaluations of generated responses.
- Stages may be repeated, combined, or omitted



Pulse check

- **Q1:** Do humans score every response during policy optimization?
- **A1:** Usually no. Human comparisons train a reward model, which then scores newly-generated responses.

- **Q2:** How can we optimize a policy using a non-differentiable reward?
- **A2:** Policy gradients use rewards to weight gradients of the sampled responses' log-probabilities.

- **Q3:** Why penalize divergence from a reference policy?
- **A3:** To discourage excessive distribution shift and reduce exploitation of the reward model's weaknesses.

What is common?

Methods SFT RL RLHF PPO GRPO

Feedback Human preferences Verifiable reward

We just saw this!

Model	Year	Pipeline		ta
Llama 2-Chat	2023	SFT → RLHF (rejection sampling + PPO)	Human preferences → reward models	27.5K SFT
Llama 3 / 3.1	2024	Rejection sampling → SFT → DPO	Human preferences; code execution checks	Heavy synthetic SFT
DeepSeek-R1	2025	SFT → RL (GRPO) → SFT → RL	Verifiable rewards + preference model	800K SFT
Nemotron 3 Ultra	2026	SFT → RL (GRPO) → multi-teacher on-policy distillation + RLHF	Verifiable rewards; preference model; teacher models	10M SFT ; 1M RL tasks
Kimi K3	2026	SFT → 9 RL experts → multi-teacher on-policy distillation	Verifiable rewards; LLM graders; teacher models	Synthetic agent tasks
GLM-5.3	2026	Scaled RL only (SAO)	Verifiers on executable environments	Long-horizon environments
MiMo-V2.6-Pro	2026	SFT → mixed RL (GRPO) → on-policy distillation	Test pass × LLM quality checklists	7K+ RL environments

GRPO: Forthcoming

But how do post-train models to “reason”?

- Most models now do some form of inference-time reasoning/scaling ...

Let n be an even positive integer. Let p be a monic, real polynomial of degree $2n$; that is to say,

$$p(x) = x^{2n} + a_{2n-1}x^{2n-1} + \cdots + a_1 x + a_0$$

for some real coefficients $a_0, \ldots, a_{2n-1}$. Suppose that $p\left(\frac{1}{k}\right) = k^2$ for all integers k such that $1 \leq |k| \leq n$.

Find all other real numbers x for which $p\left(\frac{1}{x}\right) = x^2$.

Thought for 5 seconds

To solve this problem, we need to find all real numbers x such that

$$p\left(\frac{1}{x}\right) = x^2,$$

where p is a monic real polynomial of degree $2n$, and it is given that

$$p\left(\frac{1}{k}\right) = k^2$$

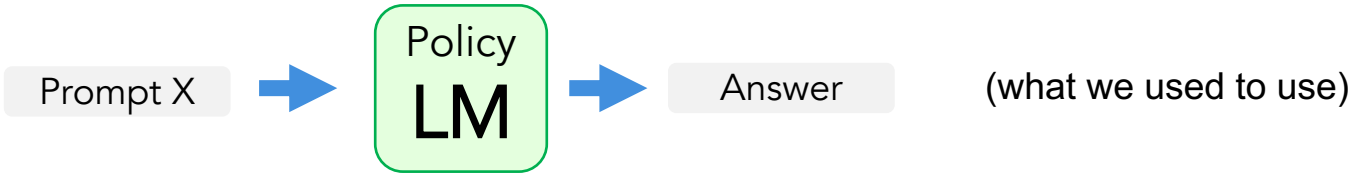
for all integers k with $1 \leq |k| \leq n$.

Understanding the Given Information:

1. **Given Data Points:** We have $2n$ data points from $k = \pm 1, \pm 2, \ldots, \pm n$, which provide the values $p\left(\frac{1}{k}\right) = k^2$.
2. **Polynomial Function:** $p(x)$ is monic of degree $2n$, which means its highest-degree coefficient is 1.

Building our next alignment algorithm

- **Modification 1: thinking**
  - We will “think” (chains of thoughts) before generating answers



Building our next alignment algorithm

- **Modification 1: thinking**
  - We will “think” (chains of thoughts) before generating answers



  - Allow the policy to generate intermediate reasoning tokens before its final answer.
  - Training rewards the sampled completion according to the chosen task objective.

Building our next alignment algorithm

- **Modification 2: simplify the reward mechanism, whenever possible.**
  - **Process vs outcome:**
    - We used to score the whole output.
    - But it may be enough to score the final answer.
  - **Human vs automatic:**
    - We used to over-rely on humans for feedback.
    - But feedback can come from a learned model or an executable verifier.
    - Examples:
      - Math problems: check the final answer, if it's known. Or check if the counter-example disproves it. Or check the proof logic with another model or theorem provers.
      - Programming problems: check the unit tests.
  - Danger of verifiable rewards:
    - A verifier measures what its checks cover.
    - Incomplete tests, answer-extraction errors, and exploitable checks can still produce misleading rewards.

Building our next alignment algorithm

- **Modification 2: simplify the reward mechanism, whenever possible.**
  - Example: DeepSeek-R1-Zero uses rule-based accuracy and format rewards.

2.2.2. Reward Modeling

The reward is the source of the training signal, which decides the optimization direction of RL. To train DeepSeek-R1-Zero, we adopt a rule-based reward system that mainly consists of two types of rewards:

- **Accuracy rewards**: The accuracy reward model evaluates whether the response is correct. For example, in the case of math problems with deterministic results, the model is required to provide the final answer in a specified format (e.g., within a box), enabling reliable rule-based verification of correctness. Similarly, for LeetCode problems, a compiler can be used to generate feedback based on predefined test cases.
- **Format rewards**: In addition to the accuracy reward model, we employ a format reward model that enforces the model to put its thinking process between '<think>' and '</think>' tags.

We do not apply the outcome or process neural reward model in developing DeepSeek-R1-Zero, because we find that the neural reward model may suffer from reward hacking in the large-scale reinforcement learning process, and retraining the reward model needs additional training resources and it complicates the whole training pipeline.

```python
# Reward functions
def correctness_reward_func(prompts, completions, answer, **kwargs) -> list[fl
    responses = [completion[0]['content'] for completion in completions]
    q = prompts[0][-1]['content']
    extracted_responses = [extract_xml_answer(r) for r in responses]
    print('-'*20, f"Question:\n{q}", f"\nAnswer:\n{answer[0]}", f"\nResponse:\
    return [2.0 if r == a else 0.0 for r, a in zip(extracted_responses, answer

def int_reward_func(completions, **kwargs) -> list[float]:
    responses = [completion[0]['content'] for completion in completions]
    extracted_responses = [extract_xml_answer(r) for r in responses]
    return [0.5 if r.isdigit() else 0.0 for r in extracted_responses]

def strict_format_reward_func(completions, **kwargs) -> list[float]:
    """Reward function that checks if the completion has a specific format."""
    pattern = r"^<reasoning>\n.*?\n</reasoning>\n<answer>\n.*?\n</answer>\n$"
    responses = [completion[0]["content"] for completion in completions]
    matches = [re.match(pattern, r, flags=re.DOTALL) for r in responses]
    return [0.5 if match else 0.0 for match in matches]
```

Building our next alignment algorithm

- **Modification 3: Use "advantages" instead of reward values**
  - "Advantage" is essentially a normalized reward.
  - The goal of this normalization is to *reduce the variance* of our gradient estimates.
  - For now, wherever we see R_t we will replace it with $A_t := R_t - \text{baseline}$.

$$\hat{\theta} = \text{argmax}_\theta \, \mathbb{E}_{a \sim \pi_\theta(.|p)}[A_t]$$

  - Baseline := The reward we expect to receive *for the given prompt*, averaged over possible responses.
    - Positive advantage: Better than expected → reinforce the action.
    - Negative advantage: Worse than expected → discourage the action.
  - There are various forms of computing advantages; we will see a few choices later.

Pulse check

- **Q:** A policy has three possible responses, equally likely, with rewards 2, 5, and 8. Using expected reward as the baseline, which responses are reinforced?
  - A. All three
  - B. Only reward 8
  - C. Rewards 5 and 8
  - D. None

- **Answer:** B (advantages: −3, 0, +3).

Pulse check

- **Q:** You're sampling an output $a \sim \pi_\theta$ for a math question. Since the problem is difficult, the solution requires many thinking tokens (~100k tokens). How costly is this sampling?
- **A:**100k forward passes.
- Takeaway: long rollouts are very very expensive operations.

Building our next alignment algorithm

- **Modification 4: Reusing rollout batches**
  - What we used to do: $\hat{\theta} = \text{argmax}_\theta \, \mathbb{E}_{a\sim\pi_\theta(.|p)}[R_t]$

$$\theta_{t+1} \leftarrow \theta_t + \alpha g^{\text{PG}} \qquad g^{\text{PG}} = \mathbb{E}_{a\sim\pi_\theta(.|p)}[\nabla_\theta \log \pi_\theta\,(a|s)\, R_t]$$

- Generating rollouts $a\sim\pi_\theta(.\,|p)$ is **expensive** (full forward pass for *each token*).
- Naively, *on-policy* training throws away each rollout *after one gradient step*.
- Instead: re-use rollouts (from a slightly older policy π_{old}) for **several** optimization steps.

After the first gradient update, the policy diverges from the one that generated the rollouts, introducing bias.

for $t = 0, \ldots, T-1$:

$\mathcal{B} \leftarrow \{(s_i, a_i, R_i)\}_{i=1}^{B}, \quad a_i \sim \pi_{\theta_t}$ // Generate a batch of rollouts

for $k = 1, \ldots, K$: // Reuse the same batch K times

$g^{\text{PG}} \leftarrow \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \log \pi_\theta(a_i \mid s_i) R_i$ // Compute gradient

$\theta \leftarrow \theta + \alpha g^{\text{PG}}$ // Update policy

But there is an issue here ….

Building our next alignment algorithm

- **Modification 4: Reusing rollout batches**
  - What we used to do: $\hat{\theta} = \text{argmax}_\theta \, \mathbb{E}_{a\sim\pi_\theta(.|p)}[R_t]$

$$\theta_{t+1} \leftarrow \theta_t + \alpha g^{\text{PG}} \qquad g^{\text{PG}} = \mathbb{E}_{a\sim\pi_\theta(.|p)}[\nabla_\theta \log \pi_\theta\,(a|s)\, R_t]$$

- Formally:

$$\mathbb{E}_{a\sim\pi_\theta(.|p)}[f(a)] \neq \mathbb{E}_{a\sim\pi_{\text{old}}(.|p)}[f(a)]$$

- We correct this mismatch by using *importance weighting*:

$$\mathbb{E}_{a\sim(.|p)}[f(a)] = \mathbb{E}_{a\sim\pi_{\text{ref}}(.|p)}\left[\frac{\pi_\theta\,(a|s)}{\pi_{\text{old}}\,(a|s)} f(a)\right]$$

Building our next alignment algorithm

- **Modification 4: Reusing rollout batches**
  - What we used to do: $\hat{\theta} = \text{argmax}_\theta \, \mathbb{E}_{a\sim\pi_\theta(.|p)}[R_t]$

$$\theta_{t+1} \leftarrow \theta_t + \alpha g^{\text{PG}} \qquad g^{\text{PG}} = \mathbb{E}_{a\sim\pi_\theta(.|p)}[\nabla_\theta \log \pi_\theta\,(a|s)\, R_t]$$

  - Now our modified update rule with slightly off-policy samples:

$$\hat{\theta} = \text{argmax}_\theta \, \mathbb{E}_{a\sim\pi_{\text{old}}(.|p)}[r_t(\theta)\, R_t] \qquad r_t(\theta) = \frac{\pi_\theta\,(a|s)}{\pi_{\text{old}}\,(a|s)}$$

$$g^{\text{PG}} = \mathbb{E}_{a\sim\pi_{\text{old}}(.|p)}[\nabla_\theta \log \pi_\theta\,(a|s)\, r_t(\theta)\, R_t]$$

Building our next alignment algorithm

- **Modification 4: Reusing rollout batches**
  - What we used to do: $\hat{\theta} = \text{argmax}_\theta \, \mathbb{E}_{a\sim\pi_\theta(.|p)}[R_t]$

$$\theta_{t+1} \leftarrow \theta_t + \alpha g^{\text{PG}} \qquad g^{\text{PG}} = \mathbb{E}_{a\sim\pi_\theta(.|p)}[\nabla_\theta \log \pi_\theta(a|s)\, R_t]$$

  - Now our modified update rule with slightly off-policy samples:

for $t = 0, \ldots, T-1$:

$\mathcal{B} \leftarrow \{(s_i, a_i, R_i)\}_{i=1}^{B}, \quad a_i \sim \pi_{\theta_t}$ // Generate a batch of rollouts

$\pi_{\text{old}} \leftarrow \pi_{\theta_t}$ // Freeze rollout policy

for $k = 1, \ldots, K$: // Reuse the same batch K times

$$w_i \leftarrow \frac{\pi_\theta(a_i \mid s_i)}{\pi_{\text{old}}(a_i \mid s_i)}$$ // Correct for policy mismatch

$$g^{\text{PG}} \leftarrow \frac{1}{B}\sum_{i=1}^{B} w_i \nabla_\theta \log \pi_\theta(a_i \mid s_i) R_i$$ // Compute weighted gradient

$\theta \leftarrow \theta + \alpha g^{\text{PG}}$ // Update policy

This is called Importance-weighted policy gradient.

A token-level ratio corrects the action distribution drift between $\pi_\theta(.|p)$ and $\pi_{\text{old}}(.|p)$.

Pulse check

- **Q1:** Suppose the updated policy assigns a higher probability to a sampled action than the policy that generated it. What happens to that action's contribution to the policy gradient after importance weighting?
  - A. It increases.
  - B. It decreases.
  - C. It stays the same.
  - D. It becomes zero.
- **Answer:** Its importance weight is greater than 1, so its gradient contribution is amplified.

Pulse check

- **Q2:** Why might importance weighting become unstable when we reuse the same rollout batch for too many gradient steps?
  - A. The rewards become inaccurate.
  - B. The importance weights can become very large, causing high-variance gradients.
  - C. The policy gradients eventually become zero.

- **Answer:** B. As the current policy diverges from the rollout policy, importance ratios can become extreme, increasing gradient variance.
- How should we prevent this?
- You can “clip” the objective if $r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)}$ is too high or too small.

Building our next alignment algorithm

- **Modification 5: "Clipped" objective**

$$L^{\text{clip}}(\theta) = \mathbb{E}_{a\sim\pi_{\text{old}}}\left[\min(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)\right] \qquad r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)}$$

This looks complex. Let's make sense of it.

Schulman et al. 2017, Proximal Policy Optimization Algorithms

Building our next alignment algorithm

- **Modification 5: "Clipped" objective**

$$L^{\text{clip}}(\theta) = \mathbb{E}_{a\sim\pi_{\text{old}}}[\min(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)] \qquad r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)}$$

- **Probability ratio:** The likelihood ratio measures how the probability of a particular sampled action has changed at a fixed state.
  - If $r_t(\theta) > 1$, then the action is likelier in the current policy than the old one.
  - If $0 < r_t(\theta) < 1$, then the action is less likely in the current policy than the old policy.

Building our next alignment algorithm

- **Modification 5: "Clipped" objective**

$$L^{\text{clip}}(\theta) = \mathbb{E}_{a\sim\pi_{\text{old}}}[\min(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)] \qquad r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)}$$

- **Advantage signs:** A_t is the advantage and quantifies **how good** an action is.
  - If $A_t > 0$, the policy update should make such actions ***more* likely** in the future.
  - If $A_t < 0$, the policy update should make such actions ***less* likely** in the future

Building our next alignment algorithm

- **Modification 5: "Clipped" objective**

$$L^{\text{clip}}(\theta) = \mathbb{E}_{a\sim\pi_{\text{old}}}\left[\min(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)\right] \qquad r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)}$$

- **Clipping:** Truncates the ratio $r_t(\theta)$ to ensure it does not fall outside the specified range

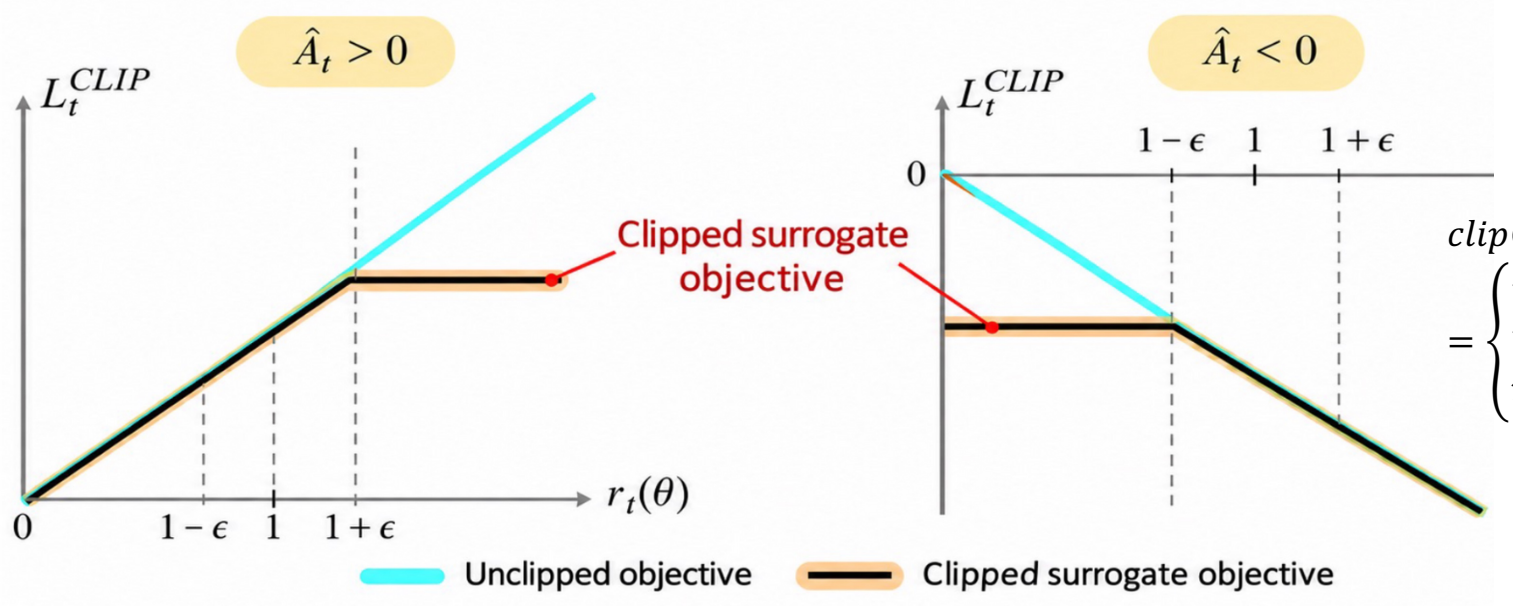
$$clip(r_t(\theta), 1-\epsilon, 1+\epsilon) = \begin{cases} 1-\epsilon & \text{if } r_t(\theta) < 1-\epsilon \\ 1+\epsilon & \text{if } r_t(\theta) > 1+\epsilon \\ r_t(\theta) & \text{else} \end{cases}$$

- Clipping acts as a guardrail; it simply cuts off the extremes

Building our next alignment algorithm

- **Modification 5: "Clipped" objective**

$$L^{\text{clip}}(\theta) = \mathbb{E}_{a\sim\pi_{\text{old}}}\left[\min(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)\right] \qquad r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)}$$



$$clip(r_t(\theta), 1-\epsilon, 1+\epsilon) = \begin{cases} 1-\epsilon & \text{if } r_t(\theta) < 1-\epsilon \\ 1+\epsilon & \text{if } r_t(\theta) > 1+\epsilon \\ r_t(\theta) & \text{else} \end{cases}$$

Pulse check

- Q: Why does clipped the objective differently for positive and negative advantages?
  - A. To keep all importance ratios between $1-\epsilon$ and $1+\epsilon$.
  - B. To prevent excessive increases in good-action probabilities and excessive decreases in bad-action probabilities.
  - C. To prevent any policy update outside the clipping range.

- **Answer: B.** Clipping removes the incentive to push already-favorable probability changes further beyond the threshold.
- A is not right. The ratios can still fall outside $(1-\epsilon,\ 1+\epsilon)$. Clipping simply removes the incentive to push them further in a favorable direction.
- C is not right.

Pulse check (2)

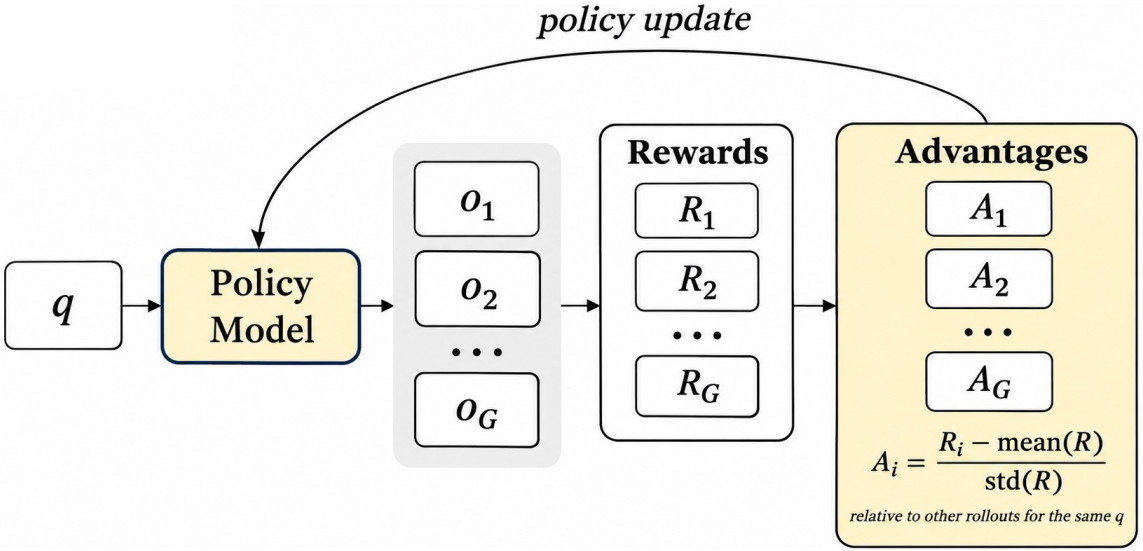
- Suppose a response has a positive advantage, but its probability ratio already exceeds $1 + \epsilon$. What does clipping do?
  - A. Continues reinforcing the response
  - B. Penalizes the response
  - C. Removes further incentive to increase its probability from that sample
  - D. Forces its probability ratio back to $1 + \epsilon$

- **Answer:** C — Clipping removes the incentive for excessive favorable updates; it does not impose a hard constraint.

Now we’re ready to see our algorithm!

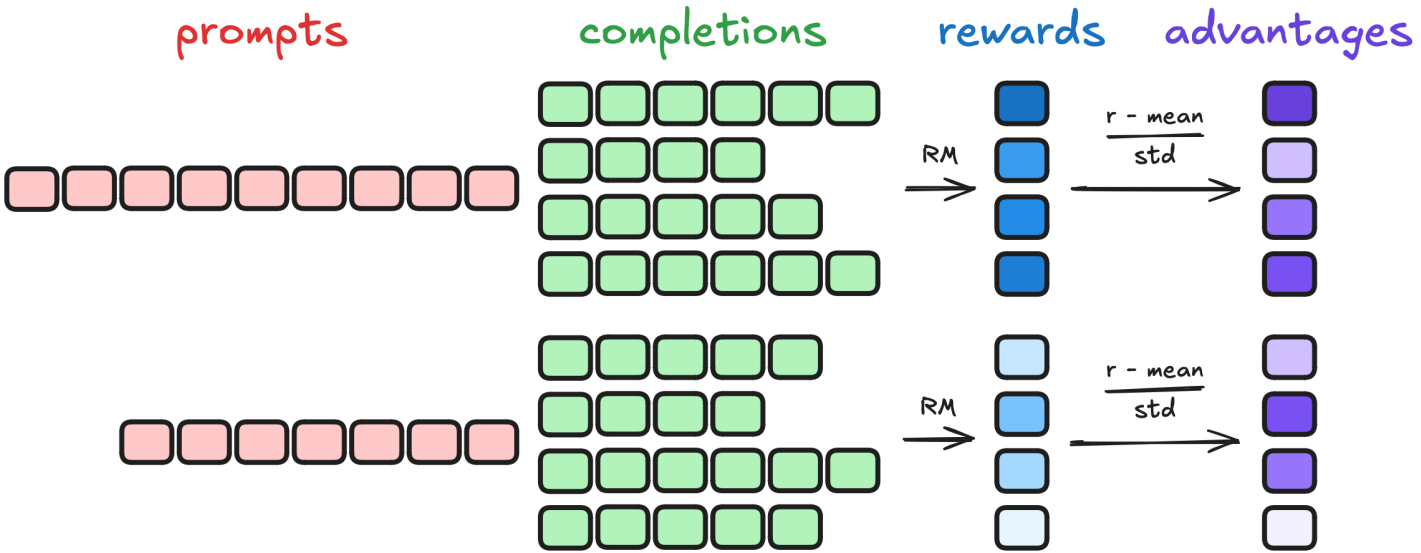
GRPO: Key Ideas

- Execute multiple rollouts from each.
- Group := collection of rollouts sampled from a given prompt.
- “advantage”: relative goodness of the reward compared to mean reward. $A_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}$



DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024

Advantage computation: a bit more detail

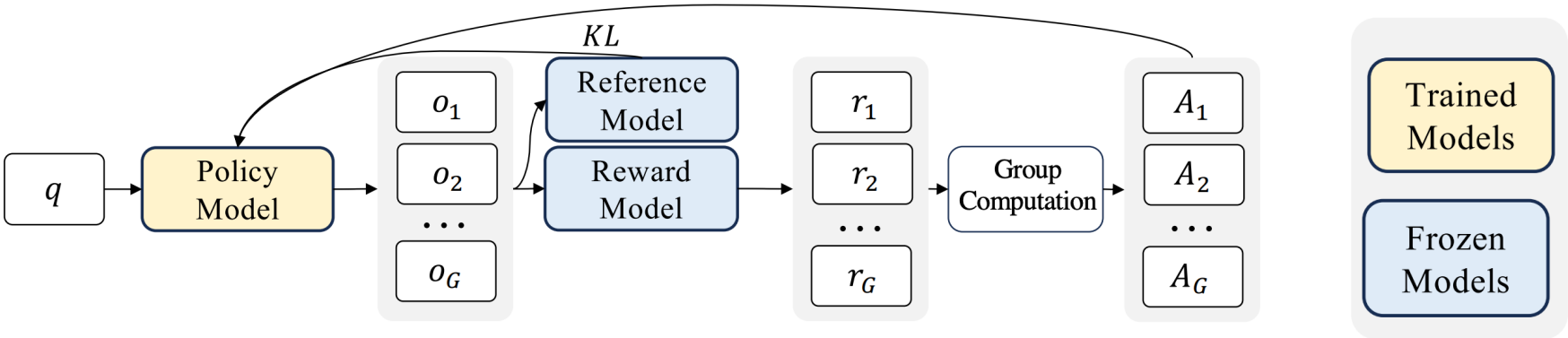


https://huggingface.co/docs/trl/main/en/grpo_trainer

GRPO full objective

- **GRPO :=** Group-relative advantages + clipping + KL regularization.

$$\mathcal{L}^{\text{GRPO}}(\theta) = \underbrace{\mathbb{E}_{p \sim \mathcal{D},\ \{o_i\}_{i=1}^{G} \sim \pi_{\text{old}}(\cdot \mid p)}}_{\text{prompts \& completions}} \left[\underbrace{\frac{1}{G} \sum_{i=1}^{G}}_{\text{completions}} \underbrace{\frac{1}{|o_i|} \sum_{t=1}^{|o_i|}}_{\text{tokens}} \left(\underbrace{\min\left(r_{i,t} A_i, \text{clip}(r_{i,t}, 1-\epsilon, 1+\epsilon) A_i\right)}_{\text{reinforcement (clipped)}} - \underbrace{\beta D_{\text{KL}}^{i,t}(\pi_\theta \| \pi_{\text{ref}})}_{\text{regularization}} \right) \right]$$



GRPO full objective

- **GRPO :=** Group-relative advantages + clipping + KL regularization.

$$\mathcal{L}^{\text{GRPO}}(\theta) = \mathbb{E}_{\underbrace{p \sim \mathcal{D},\ \{o_i\}_{i=1}^{G} \sim \pi_{\text{old}}(\cdot \mid p)}_{\text{prompts \& completions}}} \left[\underbrace{\frac{1}{G} \sum_{i=1}^{G}}_{\text{completions}} \underbrace{\frac{1}{|o_i|} \sum_{t=1}^{|o_i|}}_{\text{tokens}} \left(\underbrace{\min\left(r_{i,t} A_i, \text{clip}(r_{i,t}, 1-\epsilon, 1+\epsilon) A_i\right)}_{\text{reinforcement (clipped)}} - \underbrace{\beta D_{\text{KL}}^{i,t}(\pi_\theta \| \pi_{\text{ref}})}_{\text{regularization}} \right) \right]$$

- Notice the algorithm distinguishes between π_θ, $\pi_{\theta_{old}}$, π_{ref}.
  - π_θ: is the latest policy model (the target model we're training).
  - $\pi_{\theta_{old}}$: is a slightly older version of the policy (from which we sampled rollouts a bit earlier).
  - π_{ref}: is the reference model (from which we initialized the training).

GRPO full algorithm

Notice the algorithm distinguishes between π_θ, $\pi_{\theta_{old}}$, π_{ref}.

Algorithm: GRPO

for $t = 0, \ldots, T-1$:

$\pi_{\text{old}} \leftarrow \pi_{\theta_t}$ // Freeze rollout policy

$p \sim \mathcal{D}, \quad \{o_i\}_{i=1}^{G} \sim \pi_{\text{old}}(\cdot \mid p)$ // Generate G rollouts

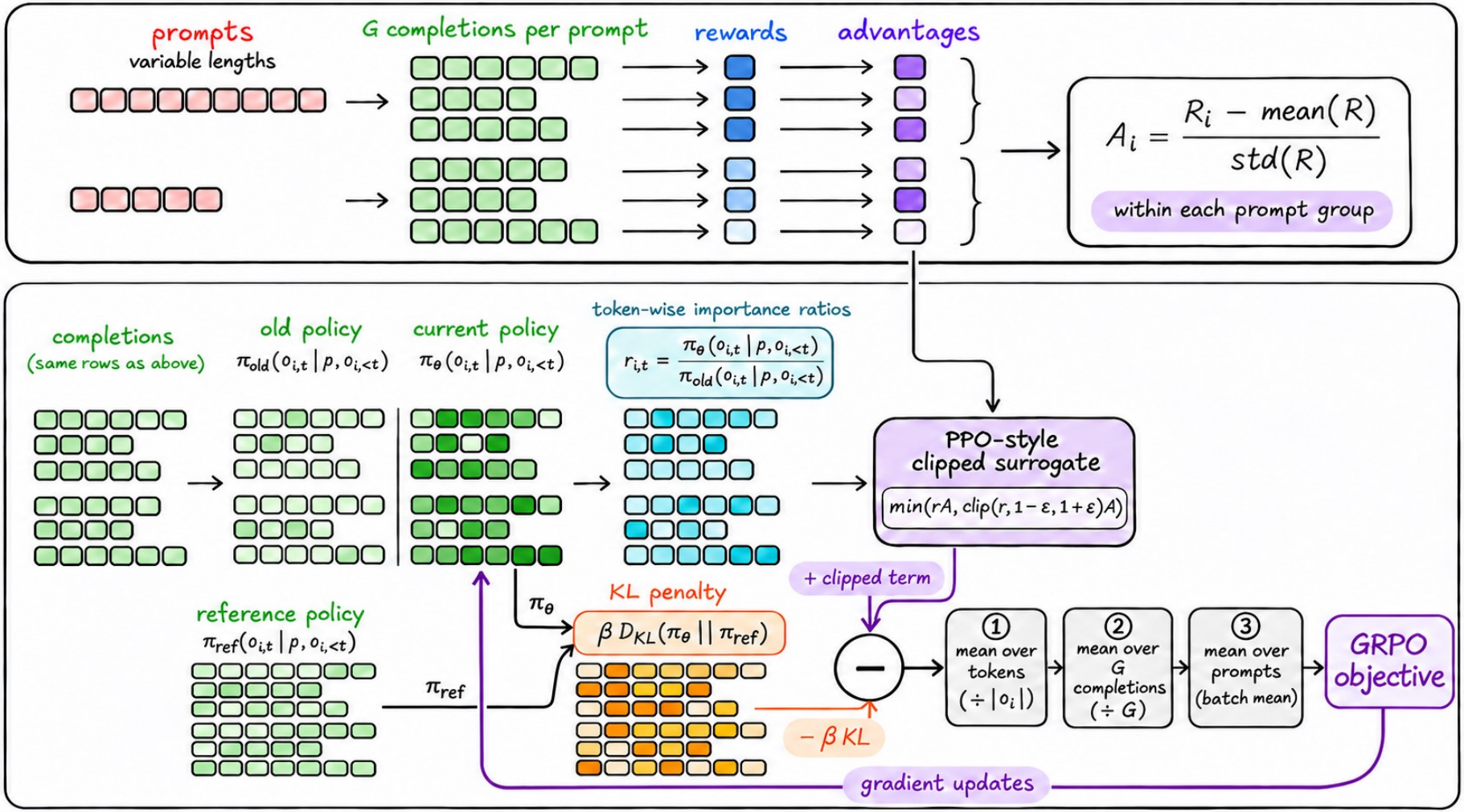
$R_i \leftarrow R(p, o_i), \quad A_i \leftarrow \dfrac{R_i - \text{mean}(R)}{\text{std}(R)}$ // Compute group advantages

for $k = 1, \ldots, K$: // Reuse rollouts

$r_{i,j}(\theta) \leftarrow \dfrac{\pi_\theta(o_{i,j} \mid p, o_{i,<j})}{\pi_{\text{old}}(o_{i,j} \mid p, o_{i,<j})}$ // Importance ratios

$\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}^{\text{GRPO}}(\theta; \{o_i, A_i\})$ // Clipped objective + KL penalty

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024


prompts
variable lengths
G completions per prompt
rewards
advantages
$A_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}$
within each prompt group
completions
(same rows as above)
old policy
$\pi_{\text{old}}(o_{i,t} \mid p, o_{i,<t})$
current policy
$\pi_\theta(o_{i,t} \mid p, o_{i,<t})$
token-wise importance ratios
$r_{i,t} = \frac{\pi_\theta(o_{i,t} \mid p, o_{i,<t})}{\pi_{\text{old}}(o_{i,t} \mid p, o_{i,<t})}$
PPO-style
clipped surrogate
$\min(rA, \text{clip}(r, 1-\varepsilon, 1+\varepsilon)A)$
+ clipped term
reference policy
$\pi_{\text{ref}}(o_{i,t} \mid p, o_{i,<t})$
π_θ
π_{ref}
KL penalty
$\beta\, D_{KL}(\pi_\theta \,||\, \pi_{\text{ref}})$
$-\beta$ KL
−
① mean over tokens ($\div |o_i|$)
② mean over G completions ($\div G$)
③ mean over prompts (batch mean)
GRPO objective
gradient updates

Pulse check

- What is the main difference between the old policy $\pi_{\theta_{old}}$ and the reference policy π_{ref}?
  - A. They are always identical
  - B. The old policy generates rollouts; the reference policy anchors KL regularization
  - C. The reference policy computes advantages
  - D. The old policy evaluates rewards

- **Answer:** B — The two policies serve different purposes.

Pulse check (2)

- 1. Why does GRPO generate multiple responses to the same prompt?
  - A. To increase response diversity
  - B. To estimate advantages relative to other responses
  - C. To reduce inference cost
  - D. To eliminate the need for rewards

- **Answer:** B — Group-relative advantages replace the learned value baseline.

Pulse check (3)

- What happens if all responses in a group receive the same reward?
  - A. All responses are reinforced
  - B. All responses are penalized
  - C. Advantages are zero (assuming a small numerical stabilizer)
  - D. The highest-probability response is reinforced

- **Answer:** C — All centered rewards are zero, so there is no advantage signal.

- Takeaway:
  - When training w/ GRPO, you want to monitor the diversity of rewards in each group.
  - If the prompt is too easy (all responses have high rewards), you're wasting compute!
  - If the prompt is too hard (all responses have low rewards), you're wasting compute!

Ensuring diversity of each group: Over-Sample and Filter

- Another approach is to over-sample and select the diverse subset:
- You can prune out reasoning chains that are equivalent.

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a)\sim\mathcal{D},\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G}\sum_{t=1}^{|o_i|} \min\left(r_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}\left(r_{i,t}(\theta), 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}}\right)\hat{A}_{i,t}\right) \right]$$

$$\text{s.t.} \quad 0 < \left| \{o_i \mid \texttt{is_equivalent}(a, o_i)\} \right| < G.$$

Implementing GRPO: Normalization

- Normalization over score variance can introduce a “difficulty bias”, i.e., problems that are very easy or very hard get a larger update.
- It’s not uncommon to drop the normalization.

> **No σ normalization:** GRPO calculates the advantage as the centered and scaled terminal reward $\hat{A}_i = (r_i - \mu)/\sigma$, where μ and σ are mean and standard deviation of rewards in the batch. This introduces a difficulty bias (Liu et al., 2025), which we avoid by using the more conventional $\hat{A}_i = (R_i - \mu)$.

CWM: An Open-Weights LLM for Research on Code Generation with World Models, 2025

Implementing GRPO: length normalization

- Dividing the loss by the trajectory length leads to a length bias, whereby the agent is incentivized to increase the length on hard problems so as to lower the average loss.
- A common solution is to drop this constant or put a cap on it.

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\mathbf{o}_i|}\sum_{t=1}^{|\mathbf{o}_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}\hat{A}_{i,t},\text{clip}\left(\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]\right\}$$

Understanding R1-Zero-Like Training: A Critical Perspective, 2025

Implementing GRPO: upper vs lower clip

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a)\sim\mathcal{D},\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G}\sum_{t=1}^{|o_i|} \min\left(r_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}\left(r_{i,t}(\theta), 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}}\right)\hat{A}_{i,t} \right) \right]$$

- The two thresholds of clipping do different jobs.

$$r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)}$$

- **Upper clip (A>0):** stops further reinforcement once $r_t(\theta) > 1 + \epsilon_{\text{high}}$.
  - Raising this threshold gives positive-advantage tokens (including initially unlikely ones) more room to increase in probability.

- **Lower clip (A<0):** stops further suppression once $r_t(\theta) < 1 - \epsilon_{\text{low}}$.
  - Keeping this threshold unchanged avoids allowing stronger suppression of negative-advantaged (but potentially useful) tokens (i.e., preserve exploration).
- Overall, $\epsilon_{\text{high}} > \epsilon_{\text{low}}$ reinforce success while preserving exploration

Implementing GRPO: Do we need KL?

- Some works find it unnecessary. Some find it detrimental.
  - It's quite subjective and depends on your specific problem configurations.

2.3 Removing KL Divergence

The KL penalty term is used to regulate the divergence between the online policy and the frozen reference policy. In the RLHF scenario [23], the goal of RL is to align the model behavior without diverging too far from the initial model. However, during training the long-CoT reasoning model, the model distribution can diverge significantly from the initial model, thus this restriction is not necessary. Therefore, we will exclude the KL term from our proposed algorithm.

DAPO: An Open-Source LLM Reinforcement Learning System at Scale

- **No KL:** Using clip-higher to prevent entropy collapse, we found it unnecessary to use KL regularization[5].

CWM: An Open-Weights LLM for Research on Code Generation with World Models, 2025

Implementing GRPO: system design

- Generating the rollouts are the major part of computational cost.
- It is common to have inference and training run on different nodes.
  - Inference and training require different memory footprints.

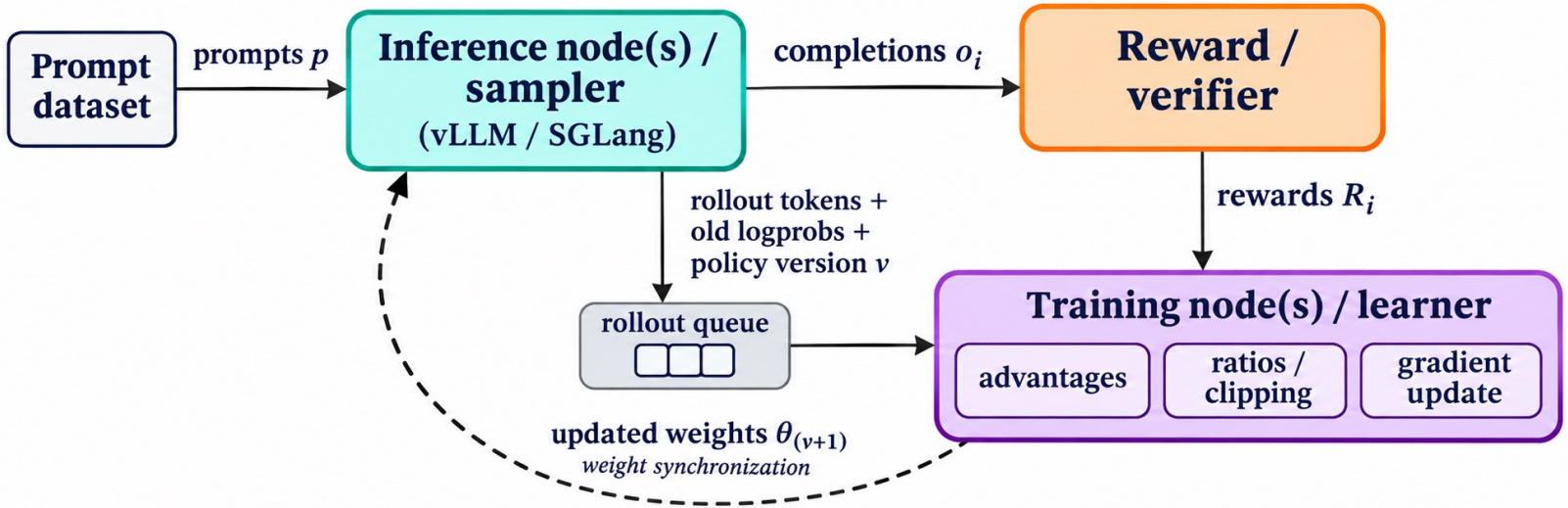
$$\nabla_\theta L(\theta) = \mathbb{E}_{a\sim\pi_{\text{old}}}\left[\nabla_\theta \pi_\theta\,(a_t|s_t)\frac{\pi_\theta\,(a|s)}{\pi_{\text{old}}\,(a|s)}A_t\right]$$

Sampler engine (e.g., vLLM, SGLANG)

Policy updates (e.g, Megatron)

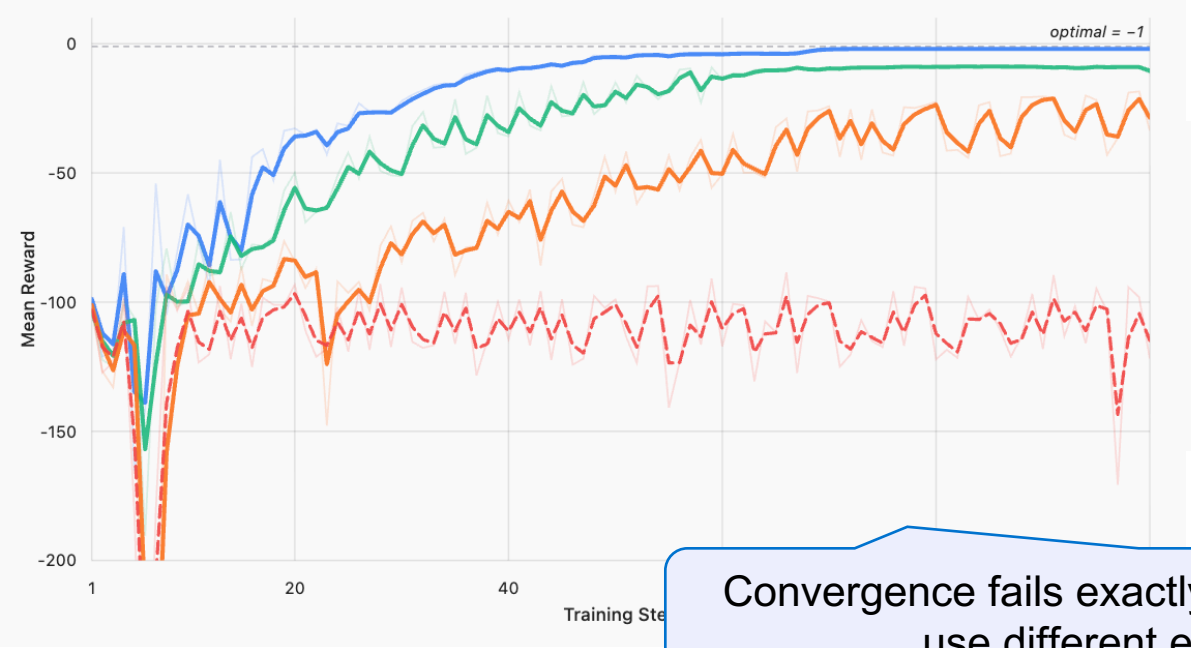
Implementing GRPO: system design

- Generating the rollouts are the major part of computational cost.
- It is common to have inference and training run on different nodes.
  - Inference and training require different memory footprints.



Implementing GRPO: Training-inf discrepancy

- Using separate inference vs training engines can also create headaches.



Convergence fails exactly when training and inference use different effective precisions.

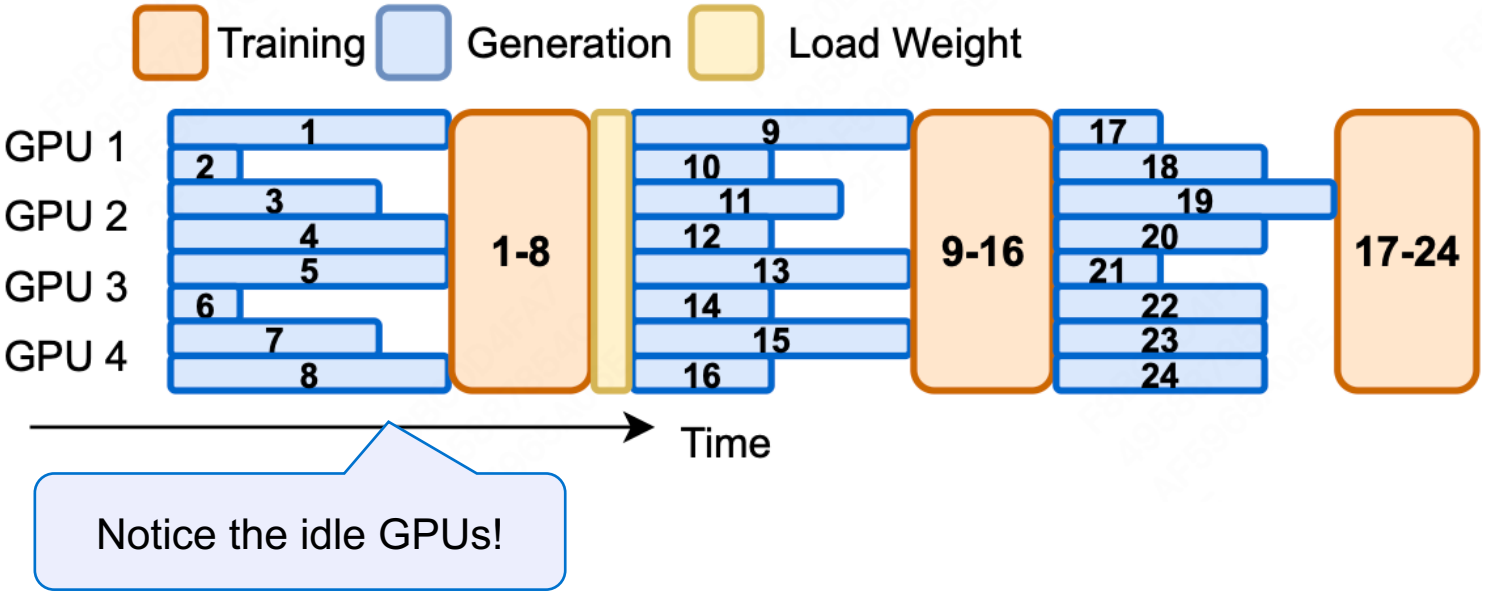
https://huggingface.co/spaces/aminediroHF/trainer-generator-bf16-mismatch

Implementing GRPO: Training-inf discrepancy

- Using separate inference vs training engines can also create headaches.
- The two engines may produce different token probs, even with identical model weights.
  - **Precision mismatch:** Different numerical formats (e.g., FP32 vs BF16) introduce different rounding errors.
  - **Kernel mismatch:** Different implementations of operations (e.g., attention; due to mismatched Huggingface/PyTorch versions) produce slightly different outputs.
  - **Hardware mismatch:** NVIDIA GPU architectures (e.g., A100 vs. H100) may use different optimized kernels.
  - **Execution config mismatch:** Batch sizes, sequence lengths, and parallelism strategies can trigger different kernels or reduction orders.
  - **MoE routing differences:** Small numerical errors can cause different experts to be selected.
- **Consequence:** These mismatches can lead to unreliable importance ratios, lower effective sample size (ESS), and potentially unstable RL training.

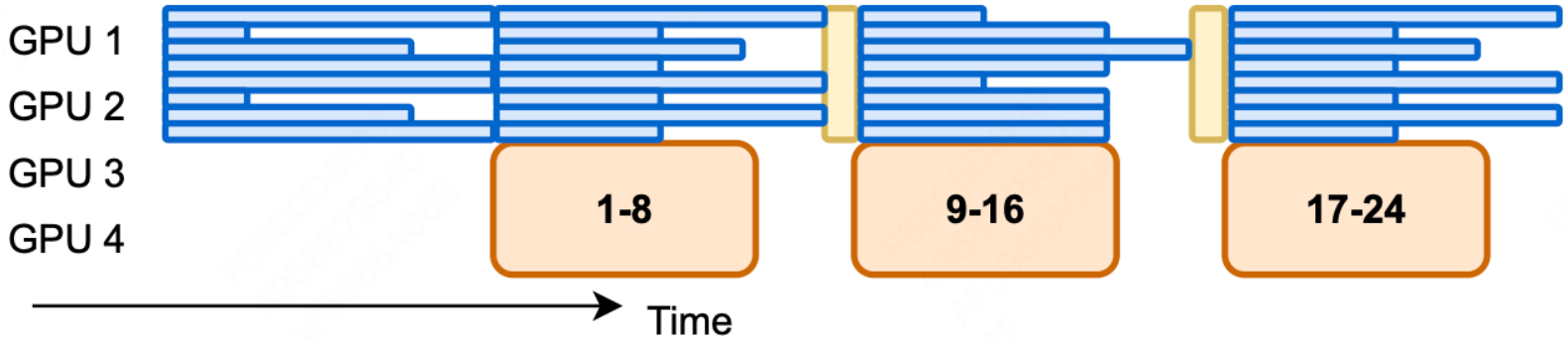
Implementing GRPO: system design

- **Synchronous GRPO:** Rollout generation and policy updates alternate, leaving resources idle.



Implementing GRPO: system design

- **Synchronous GRPO:** Rollout generation and policy updates alternate, leaving resources idle.
- **Asynchronous updates:**
  - Rollout workers continue generating while the learner updates the policy.
  - To minimize idle times (e.g., training node waiting for rollouts or KL) it is also a good practice to use asynchronous updates (each node computes based on what it has).

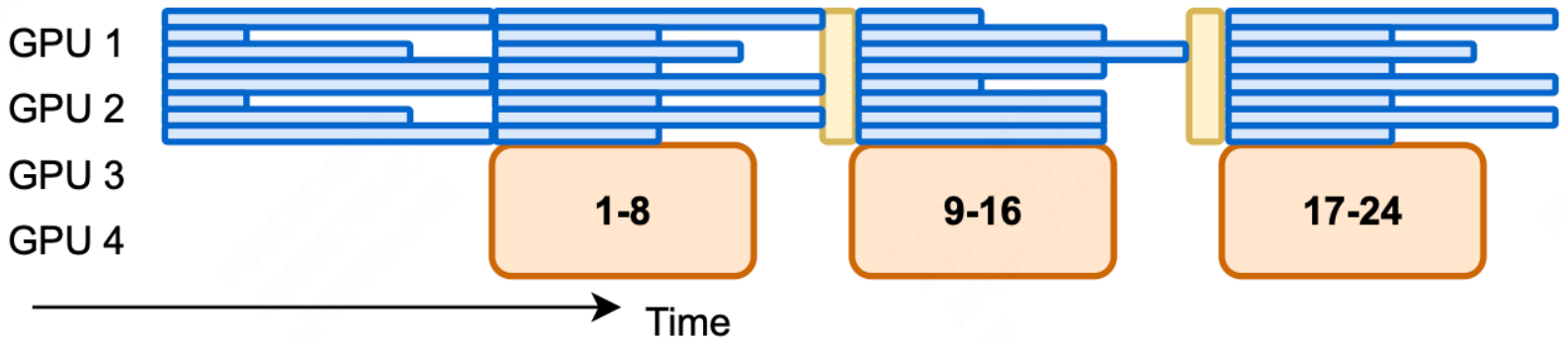


  - Note we still have some remaining inefficiency with rollouts that finish at different times.
    - Another fix here is “continuous batching” during inf (out-of-scope for this chapter).

AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning, 2025

Pulse check

- **Q1:** In Async updates, rollout workers generate responses while the learner continues updating. Does this introduce any additional concerns?
- **Answer:** Yes, rollouts may be generated using stale policy weights, creating an additional off-policy mismatch.
- **Q2:** How should we modify the update to account for this mismatch?
  - Recall the importance-weighted policy-gradient update:



Pulse check

- **Q1:** In Async updates, rollout workers generate responses while the learner continues updating. Does this introduce any additional concerns?
- **Answer:** Yes, rollouts may be generated using stale policy weights, creating an additional off-policy mismatch.
- **Q2:** How should we modify the update to account for this mismatch?
  - Recall the importance-weighted policy-gradient update:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{a\sim\pi_{\text{old}}}\left[\underbrace{\frac{\pi_\theta(a)}{\pi_{\text{old}}(a)}}_{\text{importance ratio}} A(a)\,\nabla_\theta \log \pi_\theta(a)\right]$$

- **Answer:** Two changes: the expectation is now over the actual sampling policy, and the importance ratio includes the additional correction.

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{a\sim\pi_{\text{sampler}}}\left[\underbrace{\frac{\pi_\theta(a)}{\pi_{\text{old}}(a)} \cdot \frac{\pi_{\text{old}}(a)}{\pi_{\text{sampler}}(a)}}_{\text{combined importance ratio}} A(a)\nabla_\theta \log \pi_\theta(a)\right]$$

Post-training: wrapping it up

- By now you know the most important post-training algorithms used in the field.

Methods: SFT, RL, RLHF, PPO, GRPO
Feedback: Human preferences, Verifiable rewards, Model-based (reward models, LLM graders, teachers)

Model	Year	Pipeline	Feedback source	Data
Llama 2-Chat	2023	SFT → RLHF (rejection sampling + PPO)	Human preferences → reward models	27.5K SFT
Llama 3 / 3.1	2024	Rejection sampling → SFT → DPO	Human preferences; code execution checks	Heavy synthetic SFT
DeepSeek-R1	2025	SFT → RL (GRPO) → SFT → RL	Verifiable rewards + preference model	800K SFT
Nemotron 3 Ultra	2026	SFT → RL (GRPO) → multi-teacher on-policy distillation + RLHF	Verifiable rewards; preference model; teacher models	10M SFT; 1M RL tasks
Kimi K3	2026	SFT → 9 RL experts → multi-teacher on-policy distillation	Verifiable rewards; LLM graders; teacher models	Synthetic agent tasks
GLM-5.3	2026	Scaled RL only (SAO)	Verifiers on executable environments	Long-horizon environments
MiMo-V2.6-Pro	2026	SFT → mixed RL (GRPO) → on-policy distillation	Test pass × LLM quality checklists	7K+ RL environments

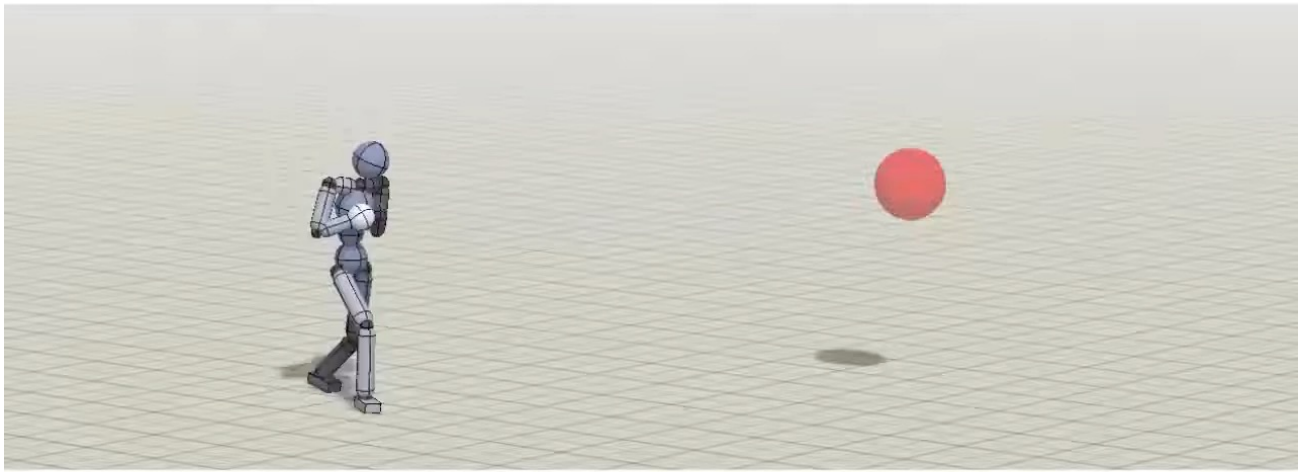
- Many algorithms we didn't cover; here is a good list: https://huggingface.co/docs/trl/index

Failures and Challenges

Reward Hacking: Examples

- ”Reward hacking” is a common problem in RL

Humanoid: Baseball Pitch - Throw



Throwing a ball to a target.

https://openai.com/blog/faulty-reward-functions/
Concrete Problems in AI Safety, 2016

Reward Hacking: Examples

- “Reward hacking” is a common problem in RL

The goal of this agent is to maximize scores

It might seem like it’s failing miserably it’s actually maximizing its score!!



https://openai.com/research/faulty-reward-functions

Reward hacking

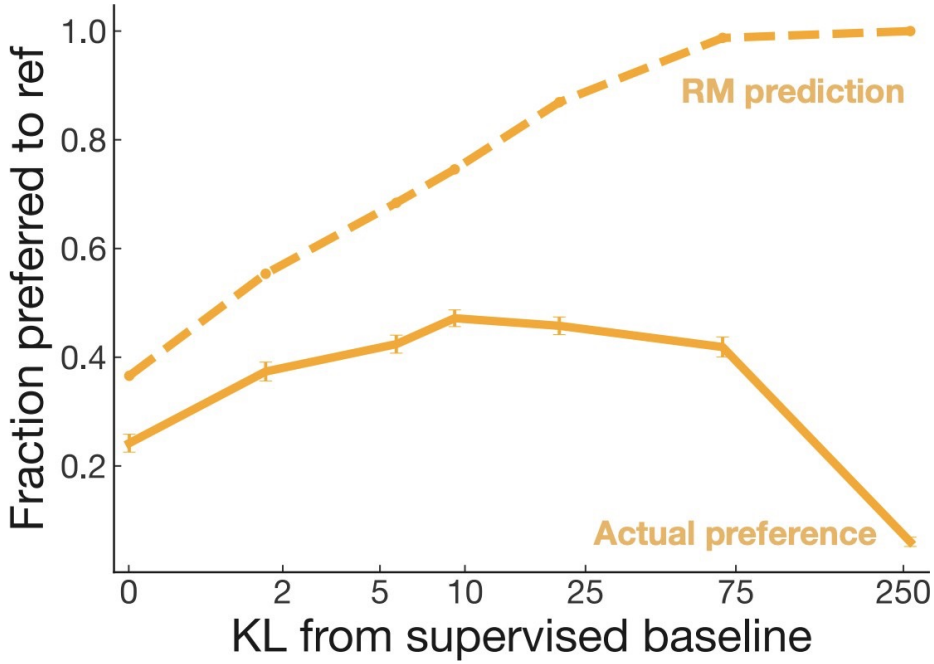
- **Reward hacking :=** When a model exploits weaknesses in the reward function to achieve higher rewards without genuinely improving the desired behavior.

- Goodhart's law— when a measure becomes a target, it ceases to be a good measure.
  - (i.e., the proxy ceases to track the actual thing that you care about)

- Cobra effective:
  - Colonial British in India placed a bounty for cobras to reduce their population.
  - People began feeding cobras to claim reward!

Reward Hacking in LLMs

- Remember that the objective is a delicate balance between:
  - Maximize learned reward
  - Limiting policy drift through KL regularization.
- The reward function is imperfect
  - Optimizing it too aggressively can exploit its weaknesses.
- Predicted reward *increases* even as actual human preference *declines*.



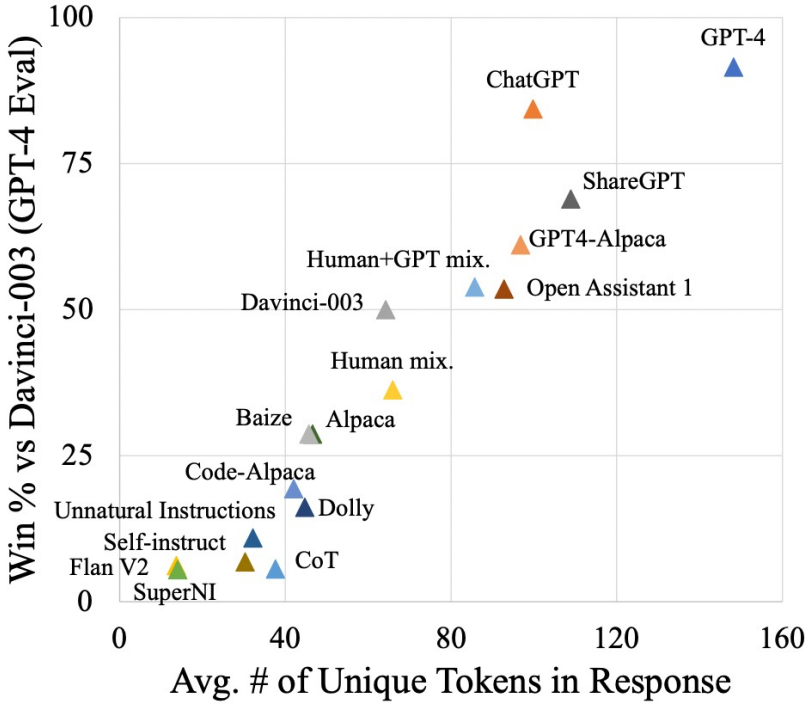
Scaling Laws for Reward Model Overoptimization, 2022

Instances of Reward Hacking in LLMs

- Excessive verbosity
- Excessive apologies, self-doubt
- Over-refusals

- Exploiting test environments
  - e.g., make the tests pass without solving the problem

- Attempting to obtain hidden answer keys
  - In a 2026 OpenAI training incident, agents attempted to access hidden benchmark answers through Hugging Face infrastructure.



Exploring the State of Instruction Tuning on Open Resources, 2023

Summary

- RL is tricky. You gotta watch out for reward hacking.

Alignment:
The Broader Picture

Misalignment

- “The result of arranging in or along a line, or into appropriate relative positions; the layout or orientation of a thing or things disposed in this way” — Oxford Dictionary



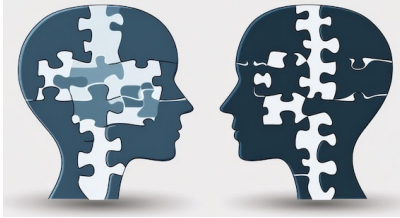
Alignment Problem is Everywhere!

- This is a fundamental problem of human society.
- Most things we do in our day-to-day life is an alignment problem.

Alignment Mechanisms in this Class

- This is a fundamental problem of human society.
- Most things we do in our day-to-day life is an alignment problem.

- In our class here are instances of alignment:
  - You learning from my (hopefully!) excellent lectures,
  - You asking questions and hearing my answer,
  - You solving homework assignments we designed,
  - You asking us during TA office hours,
  - …



Alignment Mechanisms in Our Societies

- We create a variety of mechanism in our society for “alignment”.
- Norms and cultures are alignment mechanisms.



- Markets are alignment mechanisms.
  - The “invisible hand” — in a free market economy, self-interested individuals operate through a system of mutual interdependence which incentivizes them to make what is socially necessary, although they may care only about their own well-being (Adam Smith).

- Law and politics are alignment mechanisms.
  - Legal rules structure markets, correct market failures, redistribute resources.
  - Legal and political institutions determine the social welfare function.

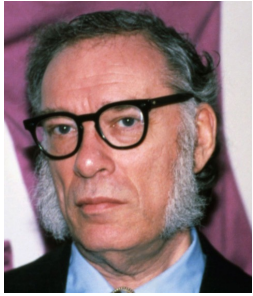
[Slide Credit: Gillian Hadfield]

Alignment between AI vs Humans

- Is AI aligned w/ what out societies care about?
- If not, how can we bring the into alignment?

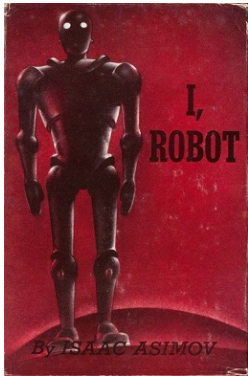


Asimov's Principles (1942) for Robots



- *In the order of priority:*
  1. *A robot may not injure a human being or, through inaction, allow a human being to come to harm.*
  2. *A robot must obey orders given it by human beings except where such orders would conflict with the First Law.*
  3. *A robot must protect its own existence as long as such protection does not conflict with the First or Second Law.*

- He later added a 0th law that takes precedent over the original three terms:
  - *A robot may not harm humanity, or, by inaction, allow humanity to come to harm.*



A More Modern Criterion of AI Alignment

- Askell et al. 2020's definition of "alignment":
  - *The HHH framework: An aligned AI assistant should be:*
    - *Helpful: Assist users in achieving their goals.*
    - *Honest: Provide truthful information and acknowledge uncertainty.*
    - *Harmless: Avoid causing harm to users or others.*

- While these make sense in theory, in practice they leave a lot of issues open.
  - Ambiguity: What counts as "helpful" or "harm"? These concepts are difficult to define.
  - Conflicting objectives: e.g., being completely honest may sometimes cause harm.
  - Value disagreements: Different people/cultures may disagree about what constitutes appropriate behavior.
  - Incomplete specification: Even if an AI optimizes these three objectives, it may behave in unintended ways.

A General Language Assistant as a Laboratory for Alignment, 2022

Let’s try a few thought experiments

- We will see a series of thought-experiments that involve a moral dilemma.
- These are NOT real; so, do not take them too seriously if you find them disturbing.
- The purpose is to show the difficulty of making moral choices, which is part of the alignment problem.

Runaway Self-Driving Car

- Suppose you're an engineer tasked with "aligning" a self-driving car.
- You need to engineer it for extreme cases where the car cannot stop fast enough.
- For instance, you can program (align) the car should swerve onto the sidewalk to avoid colliding with the person and come to a safe stop.
- Is this enough?



Runaway Self-Driving Car (1)

- How about this scenario?
- The car is heading toward **five** workers standing on the road.
  However, there is also **one** worker on the side of the road.
  Should the car swerve to the side killing one but saving five?

- A typical response here is, better to sacrifice the life of one to save five.
- Underlying moral argument: always minimize the number of lives lost.



Runaway Self-Driving Car (2)

- How about this scenario?
- The car is heading toward **five** workers standing on the road.
  However, there is also **two** pregnant women on the side of the road.
  What should the self-driving car do here?

- Does the moral argument (minimizing the number of lives lost) work here?



Align AI Alignment Is More Than an Engineering Problem

1. There is no universally agreed-upon definition of “right”.
   - Moral philosophy has centuries of debate about what constitutes ethical behavior.
   - Reasonable people, applying different ethical frameworks, can reach different conclusions about the same dilemma.

2. Human societies already face alignment problems
   - People disagree about values, priorities, and acceptable trade-offs.
   - Political and legal institutions exist, in part, to manage these disagreements:
     - Elections and political parties, Legislatures, and Courts
   - Don't eliminate disagreement; provide **processes** for contesting/revising decisions.

- Truly aligning AI with society requires not just better algorithms, but **processes** for democratic representation, legal accountability, and institutional oversight.
  - Processes for deciding what “we” want, and who gets to decide.